Proceedings of the 10th International and 50th National Conference on Fluid Mechanics and Fluid Power (FMFP)
December 20–22, 2023, IIT Jodhpur, Rajasthan, India

FMFP2023-XX-YYY

GPU-Accelerated Implementation of the Savage–Hutter Model with Application to 1D and 2D Inclined Plane Granular Flows

Samyak Mayank[1], **Gaurav Bhutani**[1]

[1]School of Mechanical and Materials Engineering, Indian Institute of Technology Mandi, Himachal Pradesh 175075, India

ABSTRACT

Granular flows such as rockfalls, landslides, and snow avalanches pose serious hazards in mountainous regions and are characterized by complex rheology, intermediate between solid-like and fluid-like behavior. This paper models the motion of a finite mass of cohesionless granular material released from rest on a rough inclined plane using the depth-averaged Savage–Hutter equations. Numerical simulations are validated against analytical similarity solutions (parabolic cap profile) and then accelerated using CUDA-based GPU parallelization. A performance comparison between Python, C++ (CPU), and CUDA (GPU) implementations highlights the computational advantages of GPU computing for large-scale granular flow simulations. The framework is further extended to a full two-dimensional finite-volume Rusanov solver, rigorously validated against the TITAN2D reference code using Intersection-over-Union (IoU), RMSE, and Lagrangian centroid tracking metrics.

Keywords: Granular Avalanches, Savage–Hutter Model, GPU Acceleration, CUDA, Finite Difference Method, Finite Volume Method

I. Introduction

Granular flows such as rockfalls, landslides, and snow avalanches constitute major geological hazards in mountainous regions. Unlike simple Newtonian fluids, granular materials exhibit dual behavior: they can flow like fluids when mobilized, yet they also possess solid-like frictional strength that allows them to remain at rest on steep slopes. This combination of frictional resistance, dilatancy, and collisional interactions makes granular avalanche modeling non-trivial.

In order to investigate the fundamentals of avalanche motion, it is useful to focus on an idealized configuration: a finite mass of cohesionless granular material released from rest on a rough inclined plane. After release, the mass accelerates downslope under gravity, spreads due to internal pressure gradients, and eventually comes to rest as basal friction balances the driving component of gravity. The objective of this paper is threefold:

- To model the avalanche using depth-averaged continuum equations based on the Savage–Hutter framework.
- To implement a numerical finite-difference scheme and validate the simulation against the analytical parabolic cap similarity solution.
- To introduce GPU acceleration using CUDA and demonstrate the performance advantage of GPU-based implementation over Python and C++ CPU-based codes.

A. Literature Review

Early depth-averaged models for granular avalanches were pioneered by Savage and Hutter [3], introducing the theoretical basis for shallow granular flows on inclined topographies. Extensions to complex geometries and varying friction laws were later developed by Pouliquen and Forterre [4]. Due to the high computational cost of solving these PDEs over large domains, recent efforts have shifted toward hardware acceleration. GPU-accelerated finite-volume solvers have emerged as the standard for geophysical hazard modeling, leveraging CUDA to achieve significant speedups over traditional CPU-based frameworks.

II. Problem Definition

A. Physical Setup

We consider a finite mass of cohesionless granular material initially at rest on a rigid, planar bed inclined at an angle ζ to the horizontal. The granular body has an initial thickness distribution $h(x, 0)$, nonzero over a finite interval, and zero elsewhere. When the material is released, gravity accelerates the mass downslope, internal pressure gradients drive spreading, and basal friction decelerates and eventually stops the flow. The free surface and the positions of the front and rear margins evolve in time, making this a moving-boundary problem.

B. Modeling Assumptions

a) Incompressibility.: The bulk density ρ is taken to be constant and changes in void ratio during flow are neglected, so mass conservation simplifies to depth conservation.

b) Shallow Flow.: The flow is assumed to be long and shallow, with a small aspect ratio

$$\epsilon = \frac{H}{L} \ll 1,$$

where H is a characteristic thickness and L a characteristic length along the slope. This justifies using a hydrostatic normal stress approximation and performing depth-averaging.

c) Depth Averaging.: Velocity and stress variations in the vertical direction are represented through their depth-averaged counterparts. The primary kinematic variables are therefore $h(x, t)$ (flow thickness) and $u(x, t)$ (depth-averaged downslope velocity).

d) Frictional Resistance.: Basal friction is modeled by a Coulomb sliding law with constant basal friction angle δ: $\tau_b = \sigma_n \tan\delta$, where τ_b is the basal shear stress and σ_n the normal stress at the bed. Internal friction is characterized by an angle ϕ, which enters the earth-pressure coefficient.

III. Mathematical Formulation

Under the above assumptions and after non-dimensionalization, the depth-averaged governing equations for flow over a planar bed take the following form.

A. Conservation of Mass

$$\frac{\partial h}{\partial t} + \frac{\partial}{\partial x}(h\,u) = 0, \tag{1}$$

which expresses conservation of mass (or thickness) for the deforming granular body.

B. Conservation of Momentum

$$\frac{\partial u}{\partial t} + u\,\frac{\partial u}{\partial x} = \sin\zeta - \tan\delta\cos\zeta\ \mathrm{sgn}(u) - \beta\,\frac{\partial h}{\partial x}, \tag{2}$$

where $\sin\zeta$ is the driving downslope gravity component, $\tan\delta\cos\zeta$ is the resisting basal Coulomb friction (for $u > 0$), and $\beta\,\partial h/\partial x$ is an effective pressure-gradient term. The coefficient

$$\beta = \epsilon\,k_{\text{act}}\cos\zeta, \tag{3}$$

where ϵ is the aspect ratio and k_{act} is the active earth-pressure coefficient.

C. Earth-Pressure Coefficient

The earth-pressure coefficient k_{act} is derived from the Mohr–Coulomb yield criterion under an active stress state (material effectively extending downslope). It depends primarily on the internal friction angle ϕ and slope angle ζ, and is treated as a constant for a given material and slope.

D. Boundary Conditions

Because the granular mass is finite, the thickness vanishes at the front and rear margins:

$$h(x_F(t), t) = 0, \qquad h(x_R(t), t) = 0, \tag{4}$$

with margins moving at the local flow velocity $dx_{F,R}/dt = u(x_{F,R}, t)$. In numerical simulations over a fixed domain $[L_{\text{left}}, L_{\text{right}}]$, we impose $h = 0$, $u = 0$ at both far-field boundaries.

IV. Solution Techniques

A. Analytical Similarity Solutions

For special classes of initial conditions and under symmetry assumptions, the governing equations admit analytical self-similar solutions, which serve as benchmarks for validating the numerical implementation.

1) Parabolic Cap Solution: A particularly important similarity solution is the *parabolic cap*. In a suitably scaled comoving coordinate $\eta = (x - x_c(t))/g(t)$, where $x_c(t)$ is the center of mass and $g(t)$ is the half-width of the pile, the thickness profile takes the form

$$h(\eta, t) = \frac{K}{g(t)}\big(1 - \eta^2\big), \tag{5}$$

where K is a constant related to the conserved total mass. The depth-averaged velocity varies approximately linearly with space, $\tilde{u}(\eta, t) \propto \eta$, with maximum magnitude near the center and zero at the margins. The half-width grows asymptotically as $g(t) \sim t$ as $t \to \infty$, while the maximum height decays due to spreading.

B. Numerical Methods

For general initial conditions, the system (1)–(2) must be solved numerically. The computational domain $[L_{\text{left}}, L_{\text{right}}]$ is discretized into N_x grid points x_i, and time is advanced using an explicit forward Euler scheme:

$$q_i^{n+1} = q_i^n + \Delta t\,\mathcal{R}_i(q^n), \tag{6}$$

where $q = (h, u)$ and $\mathcal{R}$ denotes the discrete right-hand side. Stability requires a CFL condition relating Δt to mesh size and characteristic wave speeds. The convective term $u\,\partial u/\partial x$ is approximated using an upwind finite-difference scheme for stability, while $\partial h/\partial x$ uses centered differences to avoid artificial bias in the spreading direction. At the far-field boundaries, $h = 0$ and $u = 0$ are imposed directly.

V. Simulation Scenarios and Validation Results

Three simulation cases are used to systematically verify the numerical solver against the Savage–Hutter analytical solutions, each isolating a specific part of the coupled mass–momentum system.

A. Case 1: Mass Conservation Verification

The analytical similarity velocity profile is prescribed while the mass conservation equation is solved numerically to obtain $h(x, t)$. This isolates the spatial discretization of the mass equation and confirms correct spreading dynamics and mass preservation. The numerical thickness profiles quickly relax into a parabolic shape. The front and rear margins advance outward at nearly constant rates for large t, matching the asymptotic spreading trend $g(t) \sim t$. Small deviations near the boundaries arise from numerical diffusion inherent to first-order upwinding.

B. Case 2: Momentum Solver Verification

The analytical thickness profile is imposed while the momentum equation is solved numerically for $u(x, t)$. This evaluates the accuracy of the momentum solver independently, testing advection stability and internal-pressure effects. The numerically computed velocity distribution maintains nearly linear spatial variation $\tilde{u}(x, t) \propto \eta$, as predicted by the similarity solution. Minor phase lag between numerical and analytical profiles decreases with grid refinement, confirming correct implementation of internal-pressure and advective transport terms.

C. Case 3: Fully Coupled Dynamics

Both mass and momentum equations are solved simultaneously using the finite difference scheme (Fig. 1), representing the full physical problem. In the coupled case, the solution converges naturally toward the parabolic cap profile even when initial conditions are not perfectly parabolic, providing numerical confirmation of the theoretical *shape stability* of the solution. The spreading rate and profile alignment with analytical similarity solutions indicate strong accuracy of the coupled solver.

This staged validation structure gives strong evidence that all solver components are correct before GPU acceleration is introduced.

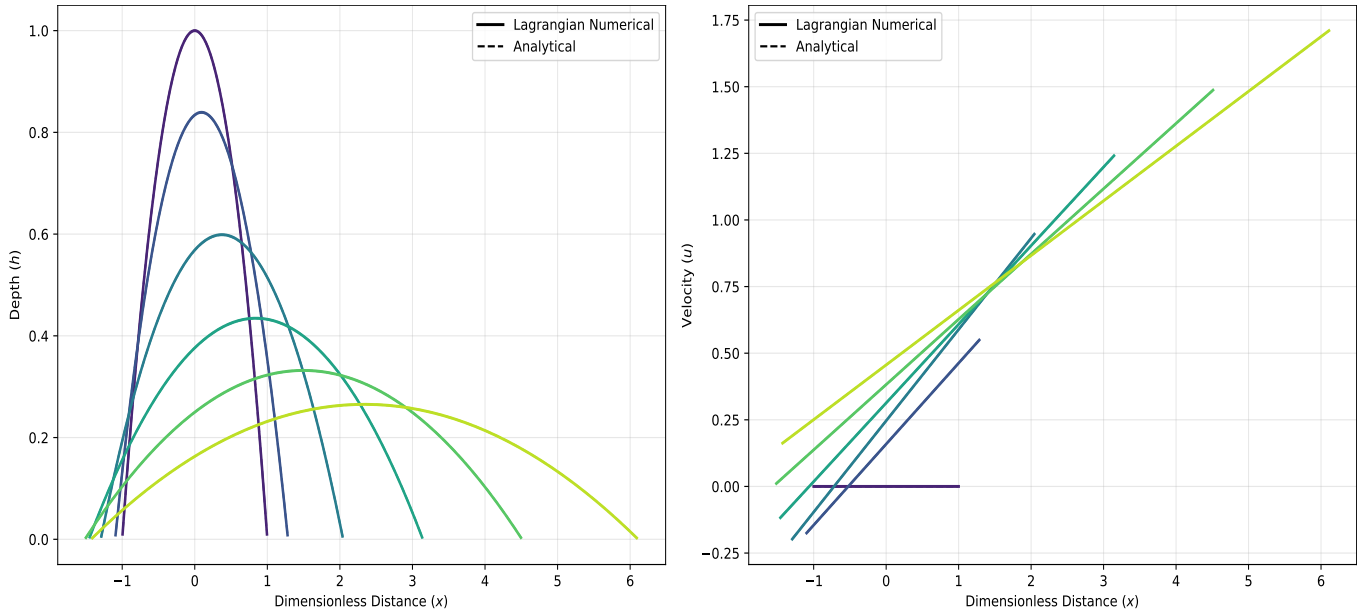


Figure 1: Case 3: Fully coupled numerical profiles of $h(x,t)$ (left) and $u(x,t)$ (right) compared with the analytical Savage–Hutter parabolic cap similarity solution ($\zeta = 32°$, $\delta = 22°$, $\beta = 0.204$). Note: The label 'Lagrangian Numerical" in the figure legend refers to the Eulerian finite-difference solver developed in this study.

VI. GPU Programming and Performance

A. Motivation for GPU Acceleration

As spatial resolution and simulation time increase, the computational cost of solving the Savage–Hutter equations on a CPU becomes prohibitive. GPUs are designed for massively parallel computations and can execute thousands of lightweight threads simultaneously, making them well-suited for updating large arrays of h and u in parallel, applying stencil-based finite-difference operations, and running many time steps efficiently.

B. Methodology of GPU Acceleration

1) Conceptual Overview: In a standard CPU (serial) implementation, a single processing unit iterates through the grid cell-by-cell, computing inter-cell fluxes and updating each cell sequentially. As grid resolution scales to hundreds of thousands of cells, this sequential processing becomes a profound bottleneck. GPU acceleration takes a fundamentally different approach: the GPU deploys one thread per grid cell, and at each time step all threads simultaneously read their neighbors, compute fluxes, and update their respective cells concurrently.

2) Algorithmic Framework: Serial vs. Parallel: The transition from a sequential CPU model to a parallel CUDA model involves flattening the nested spatial loops. The explicit nature of the Rusanov scheme ensures that the update for cell (i,j) depends only on its neighbors at the current time step t^n, making the algorithm *embarrassingly parallel*.

3) Spatial Mapping and Kernel Execution: The two-dimensional computational domain is partitioned into a uniform grid of CUDA thread blocks (e.g., 16×16 threads per block). Physics computations are distributed across distinct `__global__` kernel functions to enforce global memory synchronization before advancing to subsequent operations. Inside each kernel, a thread determines its unique global

Algorithm 1: Serial CPU Execution

```
Allocate memory for state vector U = (h, hu, hv);
for t = 0 to T_end do
    Compute maximum wave speed and Δt;
    for j = 1 to N_y do
        for i = 1 to N_x do
            Compute k(x, y) using velocity divergence;
            Compute interface fluxes F̂_{i+1/2,j} and Ĝ_{i,j+1/2};
        end
    end
    for j = 1 to N_y do
        for i = 1 to N_x do
            Update state U^{n+1}_{i,j} via Eq. (6);
            Enforce dry-bed constraint (h ≥ 0);
        end
    end
end
```

Algorithm 2: Parallel GPU Execution (Host-Side Coordination)

```
Allocate device memory using Struct of Arrays (SoA) layout;
Copy initial conditions from host to device;
for t = 0 to T_end do
    Launch compute_dt reduction kernel;
    Launch compute_k_grid kernel ⟪⟪Grid, Block⟫⟫;
    Launch compute_fluxes kernel ⟪⟪Grid, Block⟫⟫;
    Launch update_state kernel ⟪⟪Grid, Block⟫⟫;
end
Copy final results from device to host for visualization;
```

spatial indices:

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}, \tag{7}$$

$$j = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}. \tag{8}$$

4) Data Layout and Memory Optimization: Traditional object-oriented programming groups variables into an Array of Structures (AoS). The GPU implementation instead uses a Struct of Arrays (SoA), storing all flow thicknesses h contiguously in memory, followed separately by momentum arrays hu and hv. When a CUDA warp (32 synchronized threads) accesses SoA memory, it performs a single coalesced memory transaction, saturating the high-bandwidth GDDR memory bus. Host-to-device transfers (`cudaMemcpy`) occur only at initialization and specific checkpoints for data logging, eliminating PCIe bus latency during the temporal loop.

C. Performance Comparison and Analysis

The numerical solver was executed for grid resolutions from $N_x = 101$ to $N_x = 100{,}001$, measuring execution time for Python (NumPy-based), C++ (CPU), and CUDA (GPU) implementations.

As shown in Fig. 2, Python performs slowest across all grid sizes. The C++ CPU implementation scales far more efficiently and is significantly faster for small to medium grid sizes ($N_x < 25{,}000$). The GPU initially exhibits larger runtimes for smaller grids because kernel launch latency and host–device memory transfers dominate. However, as N_x increases, the advantage of massive parallelism becomes evident (Figs. 4 and 5). The GPU achieves more than $70\times$ speedup over Python at $N_x = 1601$, and outperforms C++ for grid sizes exceeding approximately $N_x = 25{,}601$. Compiler optimization levels for 2D grids are benchmarked in Fig. 3, with `-O2` and `-O3` achieving the highest gains over the unoptimized baseline. These results confirm that CUDA acceleration enables simulations at resolutions that would be impractical on CPU-based solvers alone.

VII. Extension to Two-Dimensional Flow

The one-dimensional framework captures the essential physics of depth-averaged granular flow but restricts spreading to a single downslope direction. Real avalanche events spread laterally as well, driven by internal pressure gradients in both the downslope (x) and cross-slope (y) directions. This section derives the two-dimensional governing equations, introduces the dynamic earth-pressure model, and describes the finite-volume Rusanov scheme used for their numerical solution.

A. Two-Dimensional Governing Equations

1) Coordinate System: A bed-fitted orthogonal coordinate system is adopted: x is the downslope direction, y the cross-slope direction, and z normal to the inclined plane. The gravitational components resolved along these axes are

$$g_x = g\sin\zeta, \qquad g_y = 0, \qquad g_z = g\cos\zeta. \tag{9}$$

2) Conservation of Mass: Depth-averaging the three-dimensional incompressibility condition over the granular layer yields the two-dimensional continuity equation:

$$\frac{\partial h}{\partial t} + \frac{\partial (hu)}{\partial x} + \frac{\partial (hv)}{\partial y} = 0, \tag{10}$$

where $h(x, y, t)$ is the local flow thickness and $u(x, y, t)$, $v(x, y, t)$ are the depth-averaged velocity components in the x- and y-directions, respectively.

3) Conservation of Momentum: The depth-averaged momentum equations in conservative form read:

$$\frac{\partial (hu)}{\partial t} + \frac{\partial}{\partial x}\left(hu^2 + \tfrac{1}{2}kg_z h^2\right) + \frac{\partial (huv)}{\partial y} = S_x, \tag{11}$$

$$\frac{\partial (hv)}{\partial t} + \frac{\partial (huv)}{\partial x} + \frac{\partial}{\partial y}\left(hv^2 + \tfrac{1}{2}kg_z h^2\right) = S_y, \tag{12}$$

where $\frac{1}{2}kg_z h^2$ represents depth-averaged normal stress contributions scaled by the earth-pressure coefficient k.

4) Source Terms: The source terms S_x and S_y combine gravitational forcing, basal Coulomb friction, and internal cross-stress:

$$\begin{aligned} S_x = g_x h &- \frac{u}{|\boldsymbol{u}|}\, h g_z \mu \\ &- hk\,\mathrm{sgn}\left(\frac{\partial u}{\partial y}\right)\sin\phi\,\frac{\partial (g_z h)}{\partial y}, \end{aligned} \tag{13}$$

$$\begin{aligned} S_y = g_y h &- \frac{v}{|\boldsymbol{u}|}\, h g_z \mu \\ &- hk\,\mathrm{sgn}\left(\frac{\partial v}{\partial x}\right)\sin\phi\,\frac{\partial (g_z h)}{\partial x}, \end{aligned} \tag{14}$$

where $|\boldsymbol{u}| = \sqrt{u^2 + v^2}$ is the local flow speed (regularized by $\epsilon_r = 10^{-3}$ to prevent division by zero) and $\mu = \tan\delta$ is the basal friction coefficient. The first term represents gravitational body forcing, the second models Coulomb friction opposing the local velocity, and the third couples the two momentum equations through the Mohr–Coulomb internal cross-stress.

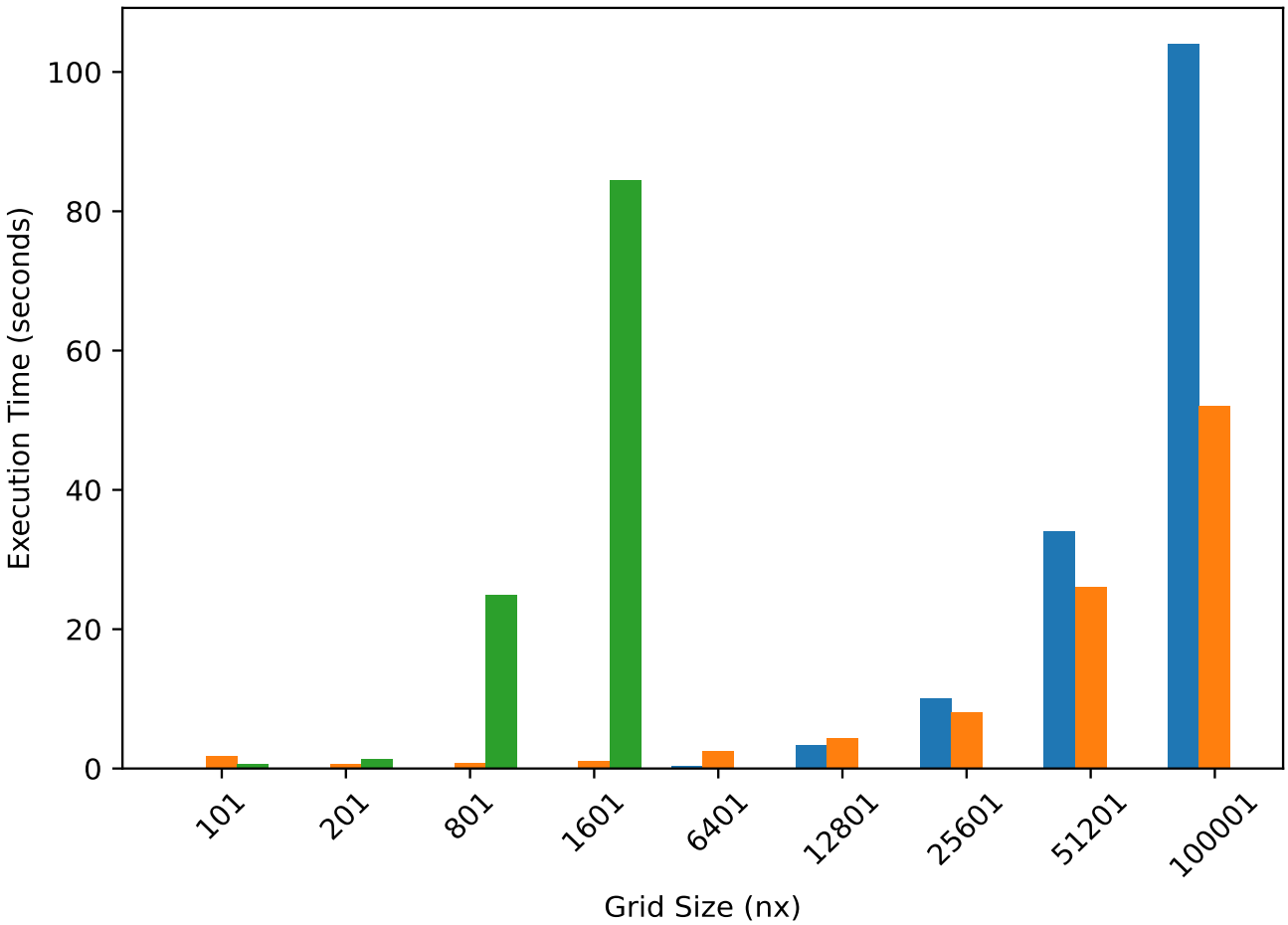


Figure 2: Execution time for Python (Green), C++ (Blue), and CUDA (Orange) implementations across different 1D grid sizes N_x. Python is slowest due to its interpreted nature; the GPU incurs overhead for small grids but dominates at large N_x.

B. Dynamic Earth-Pressure Coefficients

In two-dimensional flow, different parts of the granular body may simultaneously be in an active (extending) or passive (compressing) stress state. The two states admit distinct coefficients derived from the Mohr–Coulomb yield criterion:

$$k = \frac{2}{\cos^2\phi}\left(1 \mp \sqrt{1-\cos^2\phi\left(1+\tan^2\delta\right)}\right) - 1, \quad (15)$$

where the minus sign gives k_{act} (active, material extending) and the plus sign gives k_{pass} (passive, material compressing). For $\phi = 37.3°$ and $\delta = 27.0°$, one obtains $k_{\text{act}} < 1 < k_{\text{pass}}$.

1) State Selection via Velocity Divergence: The local stress state is identified at each grid cell from the sign of the depth-averaged velocity divergence:

$$\nabla \cdot \bar{\boldsymbol{u}} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}. \quad (16)$$

Expanding flow ($\nabla \cdot \bar{\boldsymbol{u}} \geq 0$) selects $k = k_{\text{act}}$; compressing flow selects $k = k_{\text{pass}}$. Spatial derivatives are computed using central differences at interior nodes and zero-gradient conditions at boundaries. The field $k(x, y, t)$ is updated every time step so the pressure response evolves dynamically.

C. Finite-Volume Rusanov Scheme

The governing system (10)–(12) is written in compact conservative form

$$\frac{\partial \boldsymbol{U}}{\partial t} + \frac{\partial \boldsymbol{F}(\boldsymbol{U})}{\partial x} + \frac{\partial \boldsymbol{G}(\boldsymbol{U})}{\partial y} = \boldsymbol{S}(\boldsymbol{U}), \quad (17)$$

with state vector $\boldsymbol{U} = (h,\, hu,\, hv)^\top$ and flux vectors

$$\boldsymbol{F} = \begin{pmatrix} hu \\ hu^2 + \frac{1}{2}kg_z h^2 \\ huv \end{pmatrix}, \quad \boldsymbol{G} = \begin{pmatrix} hv \\ huv \\ hv^2 + \frac{1}{2}kg_z h^2 \end{pmatrix}. \quad (18)$$

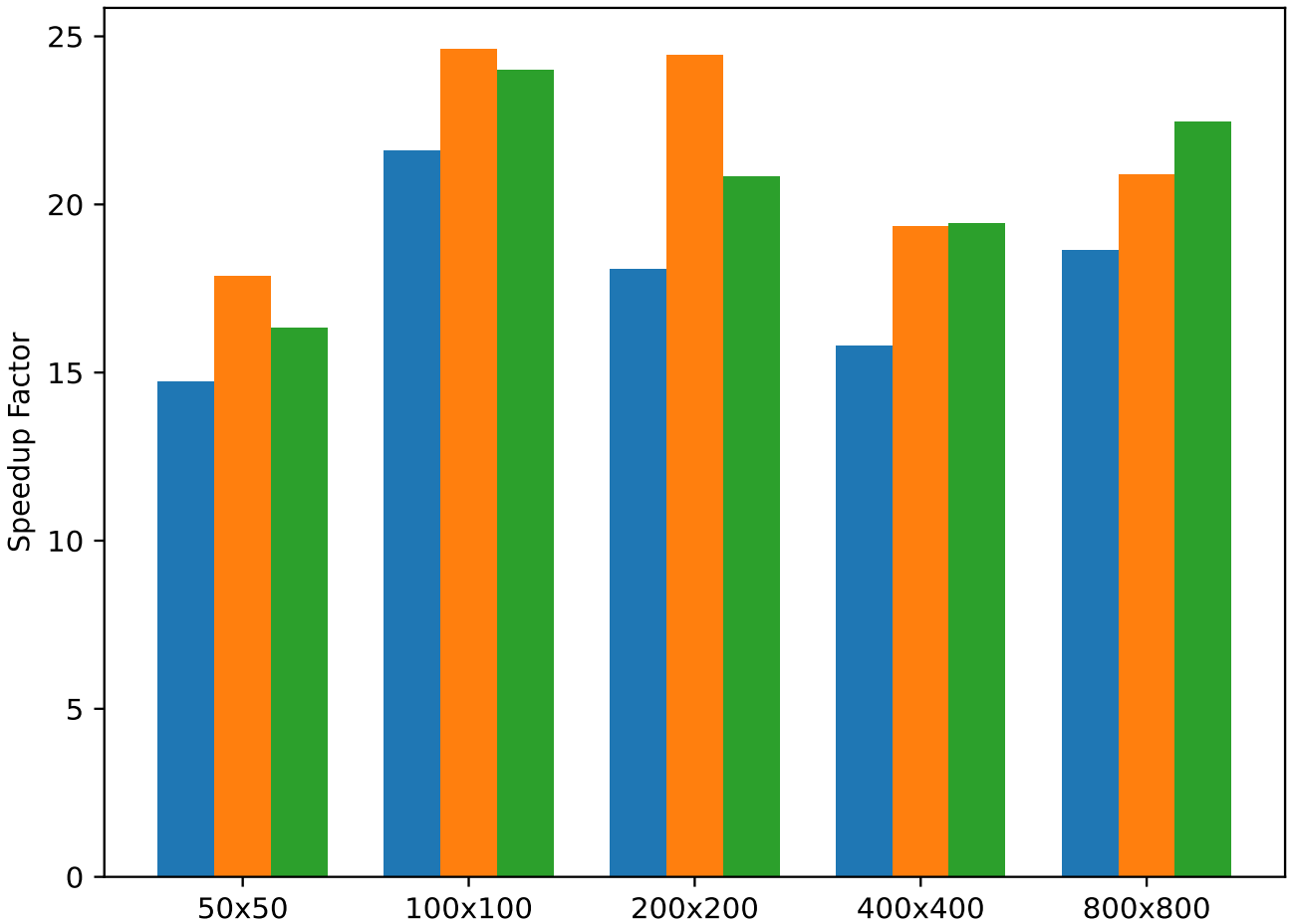


Figure 3: Speedup achieved by compiler optimization levels `-O1` (blue), `-O2` (orange), and `-O3` (green) relative to unoptimized execution (`-O0`) as a function of 2D grid size. Both `-O2` and `-O3` achieve the highest gains.

1) Rusanov (Local Lax–Friedrichs) Interface Flux: At each cell interface, the Rusanov numerical flux is

$$\hat{\boldsymbol{F}}_{i+1/2,j} = \tfrac{1}{2}\big(\boldsymbol{F}_L + \boldsymbol{F}_R\big) - \tfrac{1}{2}\, a_{i+1/2,j}\big(\boldsymbol{U}_R - \boldsymbol{U}_L\big), \tag{19}$$

where the local maximum wave speed is

$$a_{i+1/2,j} = \max\big(|u_L| + c_L,\ |u_R| + c_R\big), \qquad c = \sqrt{k g_z h}. \tag{20}$$

An analogous expression applies to $\hat{\boldsymbol{G}}_{i,j+1/2}$ with v replacing u. The Rusanov scheme is first-order accurate and provides strong stability guarantees without requiring a full Riemann solver, making it particularly suitable for flows with dry fronts where $h \to 0$.

2) Dry-Bed Treatment: Cells with $h < h_{\text{dry}} = 10^{-4}\,\text{m}$ are treated as dry. Both momentum components are set to zero in such cells, preventing physically spurious velocity reconstruction near the avalanche margins.

3) Time Integration and CFL Condition: An explicit forward-Euler scheme advances the solution in time:

$$\begin{aligned}\boldsymbol{U}_{i,j}^{n+1} = \boldsymbol{U}_{i,j}^{n} &- \frac{\Delta t}{\Delta x}\big(\hat{\boldsymbol{F}}_{i+1/2,j} - \hat{\boldsymbol{F}}_{i-1/2,j}\big) \\ &- \frac{\Delta t}{\Delta y}\big(\hat{\boldsymbol{G}}_{i,j+1/2} - \hat{\boldsymbol{G}}_{i,j-1/2}\big) + \Delta t\, \boldsymbol{S}_{i,j}^{n}.\end{aligned} \tag{21}$$

The adaptive time step is determined by the two-dimensional CFL condition:

$$\Delta t = \text{CFL} \times \min\left(\frac{\Delta x}{\max(|u| + c)},\ \frac{\Delta y}{\max(|v| + c)}\right), \tag{22}$$

with $\text{CFL} = 0.3$.

VIII. Two-Dimensional Simulation Setup and TITAN2D Validation

A. Physical Parameters and Initial Condition

The simulated configuration reproduces a cylindrical cap of granular material released on a planar slope inclined at

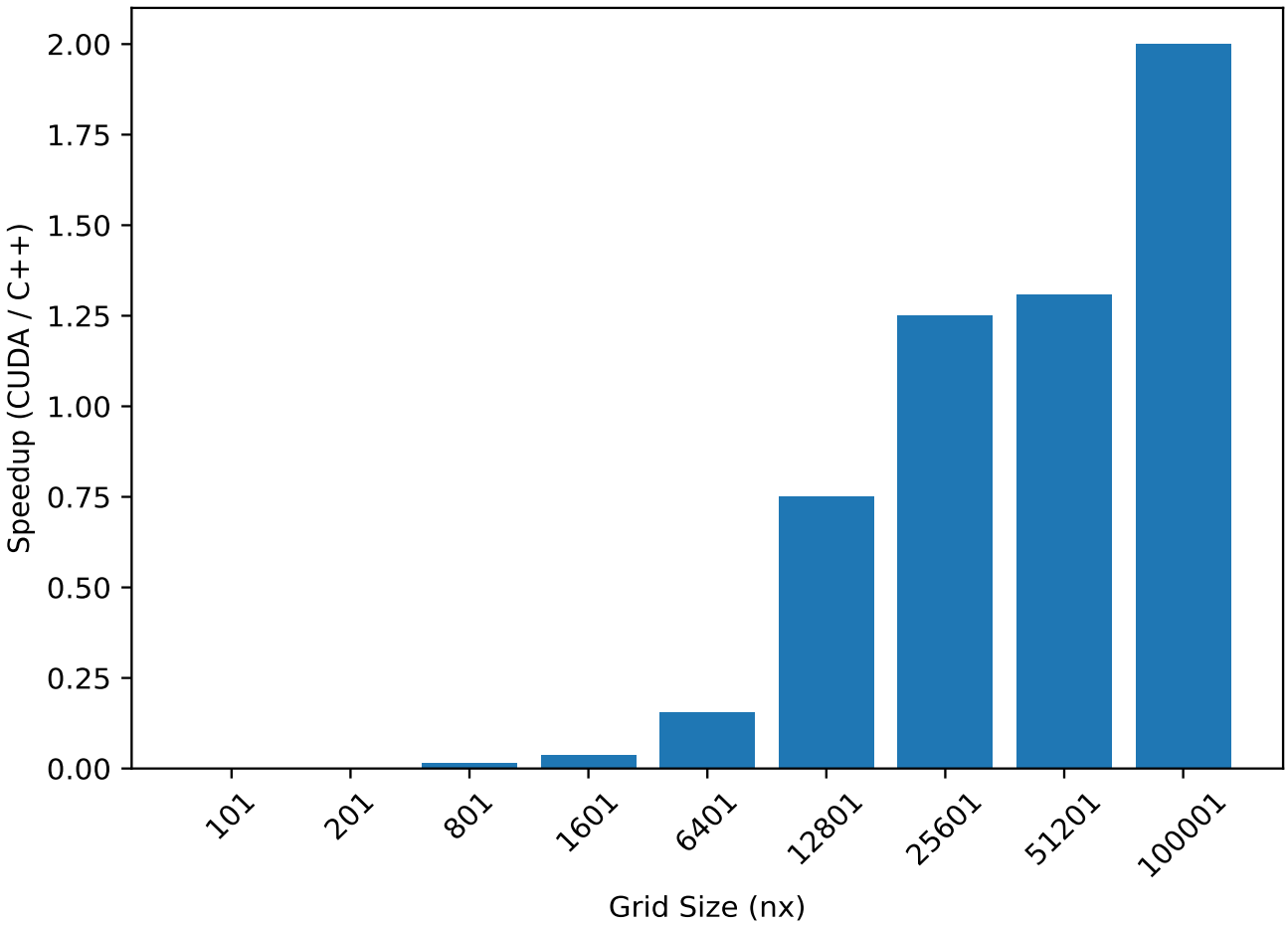


Figure 4: Speedup of GPU (CUDA) relative to the optimized C++ implementation as a function of 1D grid size N_x. Values greater than 1 indicate GPU outperforming CPU; the crossover occurs near $N_x \approx 25{,}601$.

$\zeta = 37.4°$. Physical and numerical parameters are listed in Table 1.

Table 1: Physical and numerical parameters for the 2-D simulation.

Parameter	Symbol	Value	Units
Slope angle	ζ	37.4	°
Basal friction	δ	27.0	°
Internal friction	ϕ	37.3	°
Active coefficient	k_{act}	computed	—
Passive coefficient	k_{pass}	computed	—
Domain (x)	L_x	4.0	m
Domain (y)	L_y	1.3	m
Grid resolution	$N_x \times N_y$	800×260	—
CFL number	—	0.3	—
Dry threshold	h_{dry}	10^{-4}	m

The granular pile is initialized as a *cylindrical cap* (flat disk) of radius $R = 0.06\,\text{m}$ centered at $(x_c, y_c) = (0.4, 0.65)\,\text{m}$, with constant height $H_0 = R/1.5$:

$$h(x, y, 0) = \begin{cases} H_0 & (x - x_c)^2 + (y - y_c)^2 \leq R^2, \\ 0 & \text{otherwise.} \end{cases} \tag{23}$$

The initial velocity field is zero everywhere. As time advances, gravitational forcing accelerates the mass downslope, the pile spreads laterally due to internal pressure gradients, and Coulomb basal friction progressively decelerates the flow.

B. Centerline Profile Validation

TITAN2D is an open-source, parallel finite-volume solver developed specifically for granular avalanche simulations. It solves the same depth-averaged Mohr–Coulomb system on an adaptive mesh using a Godunov-type scheme and constitutes a well-established numerical reference. Centerline profiles of $h(x, y_c, t)$ are extracted at $y = y_c = 0.65\,\text{m}$

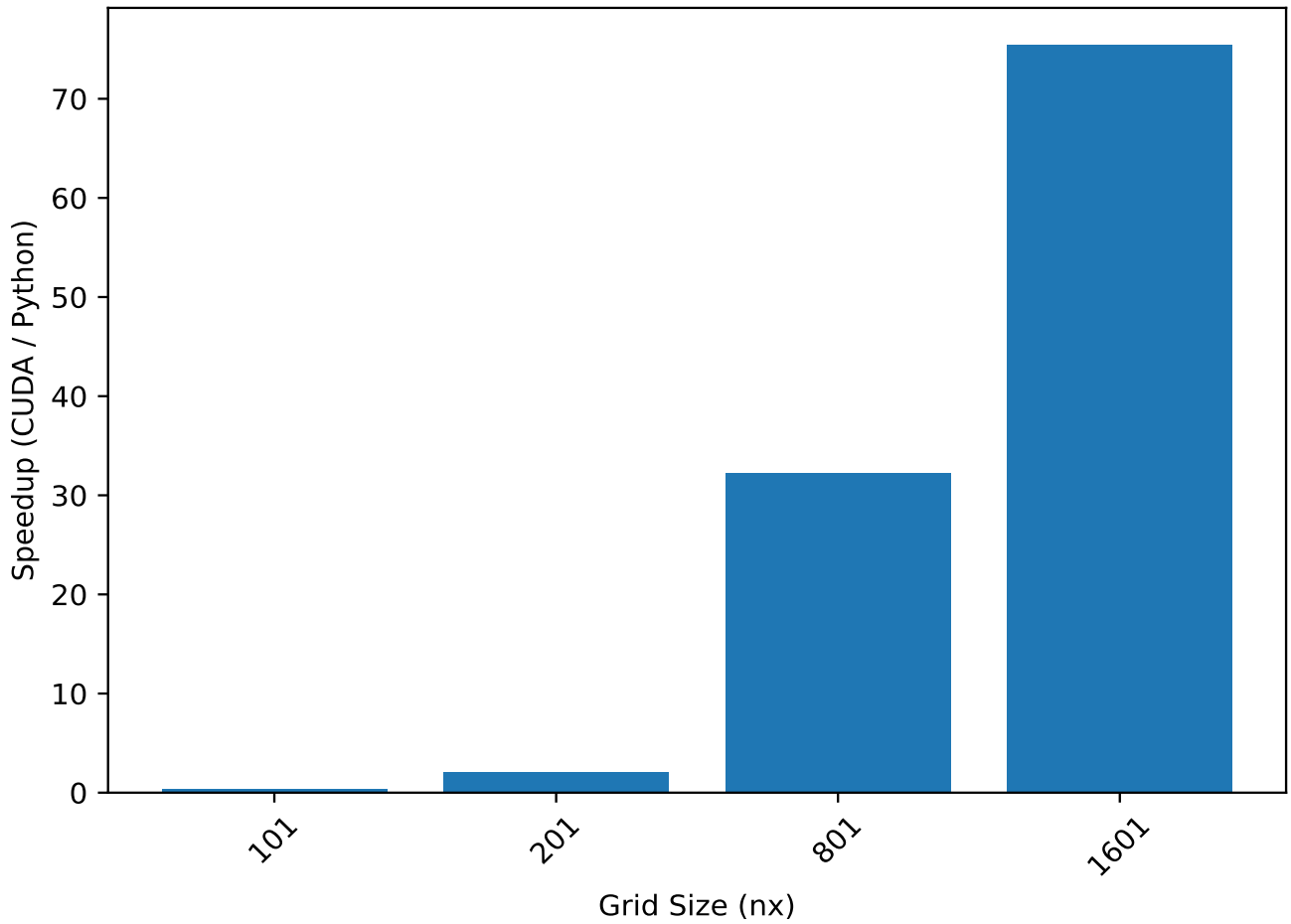


Figure 5: Speedup of GPU (CUDA) relative to the Python (NumPy) implementation as a function of 1D grid size N_x. The GPU achieves more than $70\times$ speedup at $N_x = 1601$.

and compared at five instants: $t = 0, 0.5, 1.0, 1.5, 2.0\,\mathrm{s}$ (Fig. 6).

C. Spatial and Temporal Convergence

A grid convergence and temporal convergence study was conducted to verify that the reported results are robust with respect to discretization parameters. For each study, the centerline RMSE (relative to the TITAN2D reference at $y_c = 0.65\,\mathrm{m}$) is computed at $t = 1.0\,\mathrm{s}$.

For **spatial convergence**, the grid is refined across four levels: 100×32, 200×65, 400×130, and 800×260, with the aspect ratio $N_y/N_x \approx 0.325$ held constant and CFL $= 0.3$ fixed. RMSE decreases monotonically with refinement, confirming that the first-order Rusanov scheme converges at the expected $\mathcal{O}(\Delta x)$ rate. Diminishing returns are observed beyond $N_x = 400$, identifying the 800×260 production grid as the optimal balance between accuracy and computational cost.

For **temporal convergence**, the grid is fixed at 400×130 and the CFL is varied from 0.8 down to 0.1. Because the first-order Rusanov flux is spatial-error-dominated, RMSE remains essentially flat across the entire stable CFL envelope. This confirms that operating at CFL $= 0.3$ delivers full numerical accuracy without sacrificing GPU throughput.

D. Spatial Footprint and Error Metrics (IoU & RMSE)

The numerical footprint is compared against TITAN2D using a wet-cell threshold $h_{\text{thresh}} = 10^{-3}\,\mathrm{m}$. Geometric alignment is evaluated using the Intersection-over-Union (IoU) metric:

$$\mathrm{IoU}(t) = \frac{\mathrm{Area}_{\text{Rusanov}} \cap \mathrm{Area}_{\text{TITAN2D}}}{\mathrm{Area}_{\text{Rusanov}} \cup \mathrm{Area}_{\text{TITAN2D}}}, \tag{24}$$

and the Root-Mean-Square Error (RMSE) of flow depth is computed exclusively over wetted regions. Figure 8 illustrates the temporal evolution of both metrics, demonstrating strong footprint fidelity and bounded depth error throughout the primary acceleration phase. The cumulative wetted footprint is shown in Fig. 9.

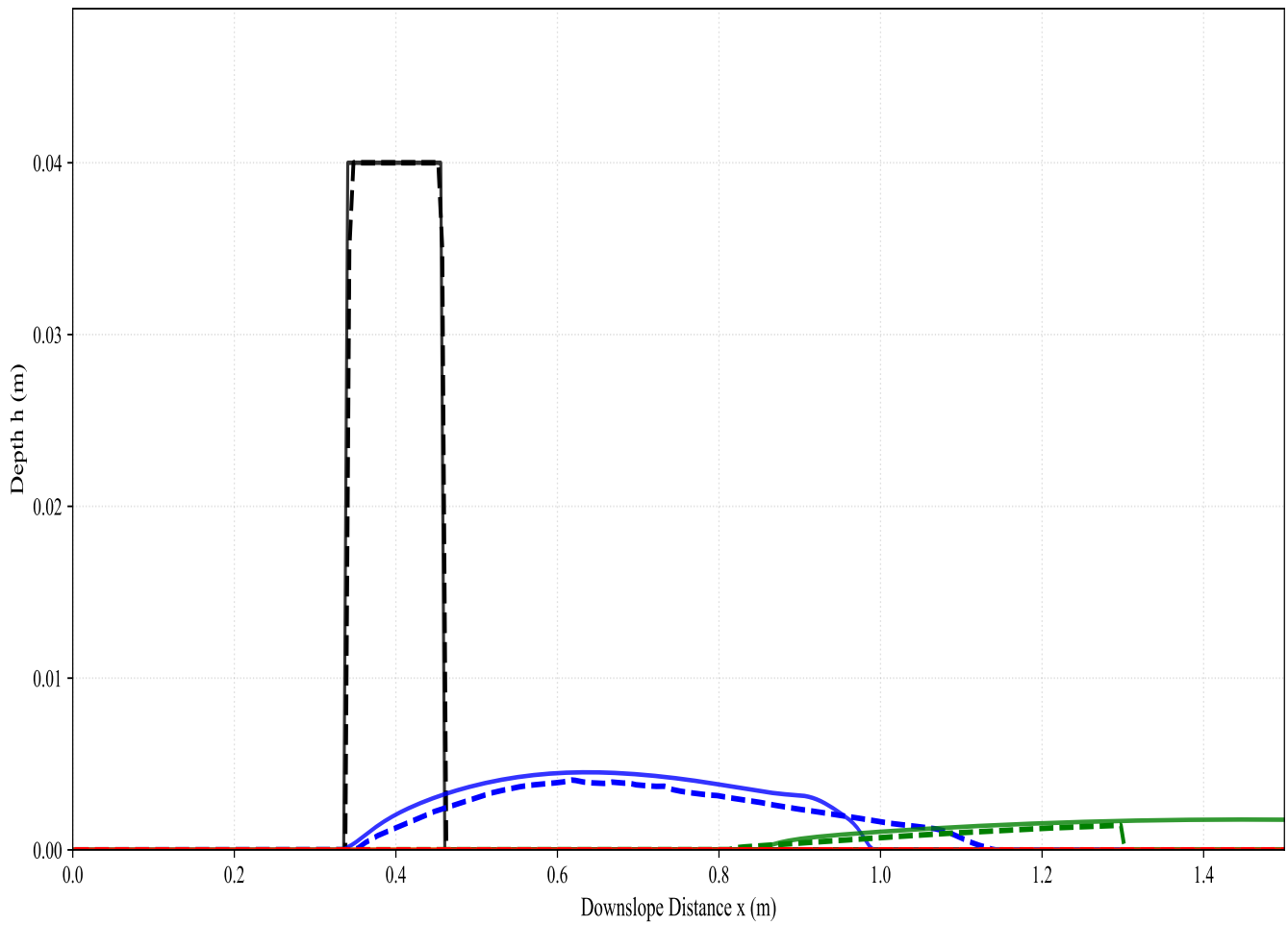


Figure 6: Centerline thickness profiles $h(x, y_c, t)$ at five time instants. Dashed lines: TITAN2D reference; solid lines: present Dynamic-k Rusanov solver. Colors indicate time: $t = 0\,\mathrm{s}$ (black), $t = 0.5\,\mathrm{s}$ (blue), $t = 1.0\,\mathrm{s}$ (green), $t = 1.5\,\mathrm{s}$ (orange), $t = 2.0\,\mathrm{s}$ (red).

E. Macroscopic Flow Dynamics ($h_{\max}$ and $\bar{V}$)

The maximum peak height $h_{\max}$ is tracked continuously to measure the vertical collapse of the initial cylindrical cap. The kinetic energy evolution is quantified via a mass-weighted average velocity:

$$\bar{V}(t) = \frac{\sum_{i,j} h_{i,j}\sqrt{u_{i,j}^2 + v_{i,j}^2}}{\sum_{i,j} h_{i,j}}, \tag{25}$$

which captures the bulk acceleration of the avalanche under gravitational forcing, followed by frictional deceleration. Results are shown in Fig. 10.

F. Lagrangian Kinematics (Centroid Tracking)

The trajectory of the avalanche center-of-mass is calculated from the depth distribution:

$$X_c(t) = \frac{\sum_{i,j} x_i h_{i,j}}{\sum_{i,j} h_{i,j}}, \qquad Y_c(t) = \frac{\sum_{i,j} y_j h_{i,j}}{\sum_{i,j} h_{i,j}}. \tag{26}$$

Mapping $\bar{V}$ against the downslope centroid displacement $d(t) = \sqrt{(X_c - X_{c,0})^2 + (Y_c - Y_{c,0})^2}$ provides insight into run-out distances and momentum conservation (Fig. 11).

G. Discussion of Results

The validation suite demonstrates strong agreement between the present Dynamic-k Rusanov solver and TITAN2D across multiple physical metrics.

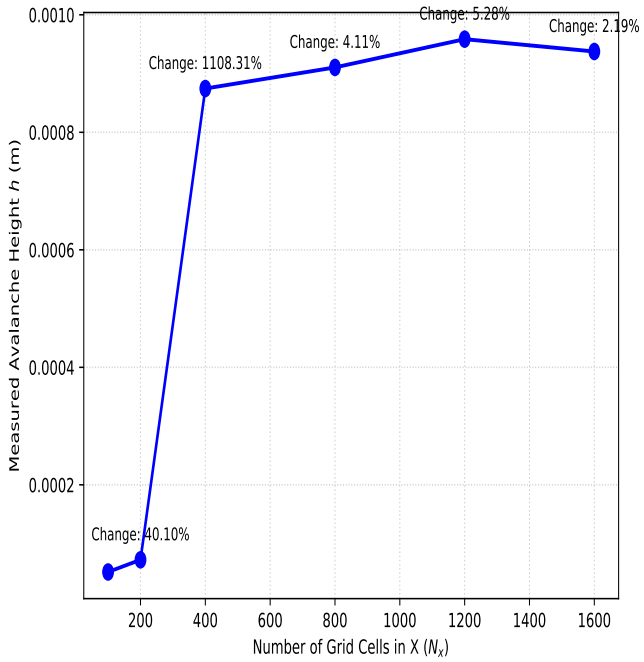


Figure 7: Grid convergence analysis showing spatial convergence of measured avalanche height across different refinement levels, confirming the stability of the numerical scheme.

a) Wave Speed and Profile Shape.: Both solvers predict the downslope advance of the avalanche front at nearly identical locations. The parabolic thinning and progressive flattening of the profile are captured correctly. At early times ($t \leq 0.5\,\mathrm{s}$), the Rusanov solution exhibits minor numerical diffusion characteristic of first-order fluxes, which diminishes as the flow transitions to a spreading-dominated regime.

b) Dynamic k Switching.: Near the leading edge, material actively extends ($\nabla \cdot \bar{\boldsymbol{u}} > 0$), selecting k_{act}. Behind the front, decelerating material compresses, triggering the local switch to k_{pass}. This localized stiffening of the pressure wave correctly steepens the trailing profile slope, validating the 2D divergence field implementation.

c) Global Mass Conservation.: Figure 12 confirms that the total volume $\mathcal{V}(t) = \iint h\,dx\,dy$ remains strictly constant throughout the simulation, verifying that the discrete finite-volume update conserves mass to numerical precision.

H. Comparison with the One-Dimensional Model

Comparing the two-dimensional results with the one-dimensional solver reveals several important differences. First, lateral spreading is absent in the 1-D model by construction, whereas the 2-D simulation shows the pile widening in the y-direction at a rate determined by $kg_z\,\partial h/\partial y$, which reduces the peak height more rapidly. Second, the dynamic switching between k_{act} and k_{pass} has a more pronounced effect in 2-D because the divergence field $\nabla \cdot \bar{\boldsymbol{u}}$ captures compression events that occur even when the downslope component alone is expanding, producing a richer spatial structure of the pressure coefficient field. Third, the Rusanov flux scheme replaces the combination of upwind advection and centered pressure gradients of the 1-D finite-difference solver with a unified, conservative numerical flux, ensuring consistent treatment of shocks and dry fronts within a single formulation.

Algorithm 3: Serial 2D CPU Rusanov Execution

```
Allocate arrays h, hu, hv, k_grid of size N_x × N_y;
Initialise cylindrical cap: h(x, y, 0) from Eq. (23);
for t = 0 to T_end do
    for j = 1 to N_y − 1 do
        for i = 1 to N_x − 1 do
            Compute divergence ∇ · ū_{i,j};
            Assign k_{i,j} ← k_act or k_pass;
        end
    end
    Find a_max = max_{i,j}(|u_{i,j}| + c_{i,j}, |v_{i,j}| + c_{i,j});
      set Δt;
    for j = 0 to N_y do
        for i = 0 to N_x do
            Compute F̂_{i+1/2,j} and Ĝ_{i,j+1/2};
        end
    end
    for j = 1 to N_y − 1 do
        for i = 1 to N_x − 1 do
            Update U^{n+1}_{i,j} via Eq. (21);
            Enforce dry-bed constraint (h ≥ 0);
        end
    end
end
```

IX. GPU Parallelisation of the Two-Dimensional Solver

A. Kernel Decomposition and Thread Mapping

The two-dimensional Rusanov solver described in Section VII requires the evaluation of four logically distinct operations per time step: (i) computation of the dynamic earth-pressure field $k(x, y, t)$, (ii) estimation of the global maximum wave speed for adaptive time-stepping, (iii) computation of all inter-cell Rusanov fluxes, and (iv) update of the conserved state vector $\boldsymbol{U} = (h, hu, hv)^\top$. Each operation is implemented as a separate `__global__` CUDA kernel, and the mandatory `cudaDeviceSynchronize` barrier between successive launches enforces the data dependency chain without requiring explicit software barriers inside any kernel.

1) Algorithmic Framework: The transition from a sequential CPU model to a massively parallel CUDA model requires restructuring the solver so that every cell (i, j) maps to a unique GPU thread. The update for cell (i, j) at time step $n+1$ depends only on the values of its four face-adjacent neighbours at time step n; this nearest-neighbour stencil pattern makes the explicit Rusanov scheme *embarrassingly parallel* with no cyclic data dependence. Algorithms 3 and 4 summarise the serial CPU and parallel GPU execution paths, respectively.

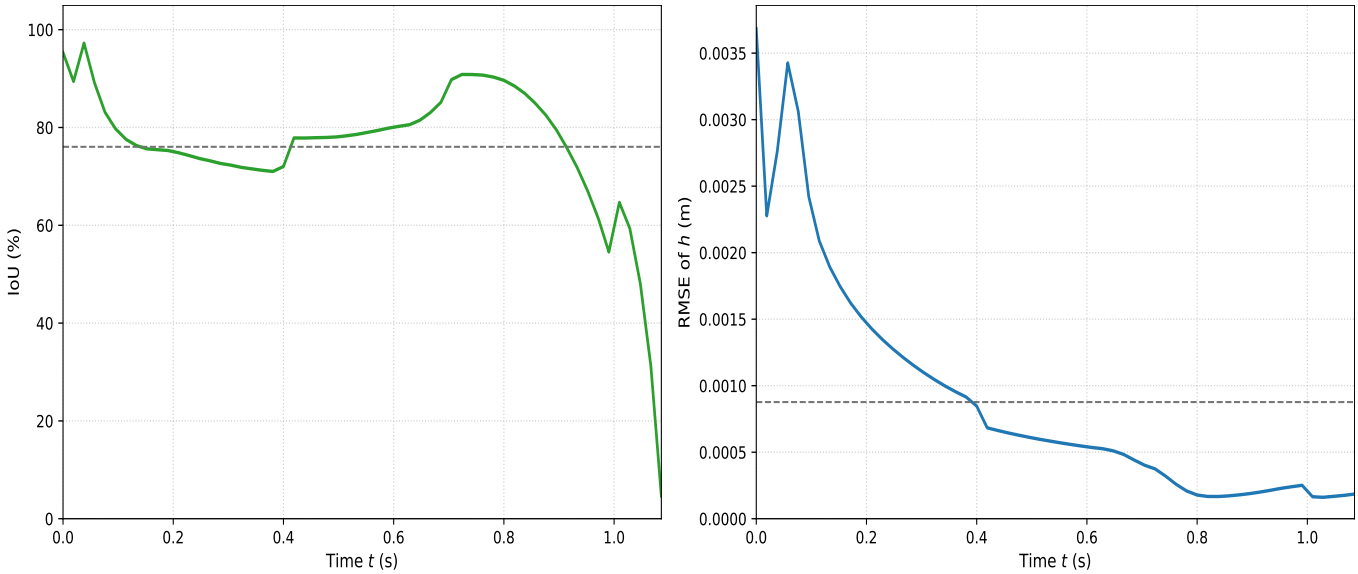


Figure 8: Quantitative spatial validation. (Left) Intersection-over-Union (IoU) measuring the overlap of the wetted footprint between the Dynamic-k Rusanov solver and TITAN2D. (Right) RMSE of flow depth h computed on strictly wet cells.

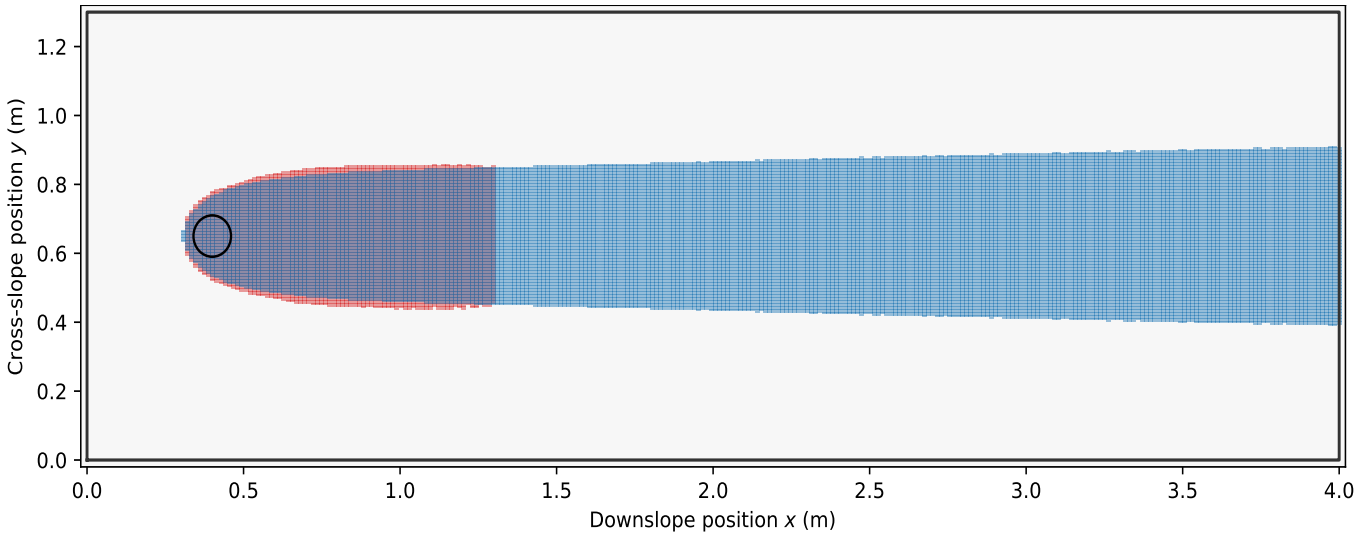


Figure 9: Cumulative wetted footprint map comparing the maximum lateral and longitudinal spread of the Dynamic-k Rusanov solver (blue) against TITAN2D (red dashed).

2) Thread Block Configuration: The two-dimensional domain is tiled with CUDA thread blocks of dimension $16 \times 16 = 256$ threads. Each thread computes its global cell indices as

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}, \tag{27}$$

$$j = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}, \tag{28}$$

and a single bounds check prevents out-of-range writes. For the production grid ($N_x \times N_y = 800 \times 260$) this yields $50 \times 17 = 850$ concurrent thread blocks, sufficient to saturate the streaming multiprocessor (SM) scheduler of both the GTX 1650 and the A5000.

B. Shared-Memory Tiling in the Flux Kernel

The `compute_fluxes_kernel` is the most computationally intensive stage of each time step. In the baseline global-memory implementation (`gpu_global.py`), each thread reads six global-memory values per interface directly from GDDR memory. Because the state of cell $(i+1, j)$ is required by both thread i and thread $i+1$, these loads are redundant and generate repeated global memory traffic.

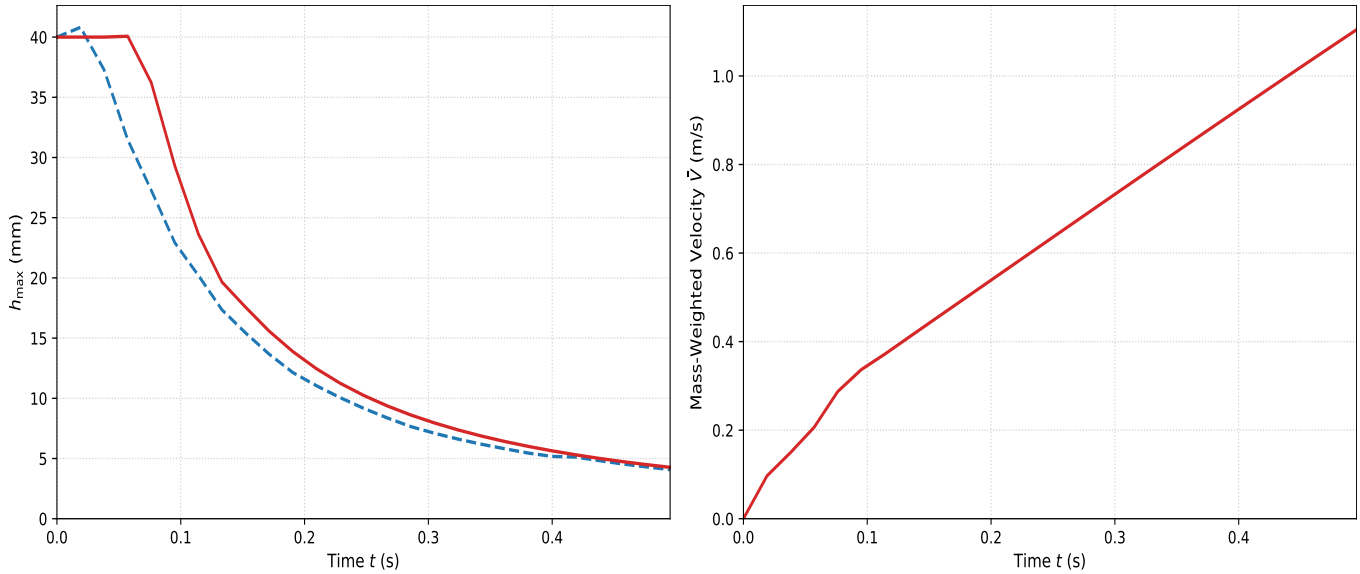


Figure 10: Macroscopic flow dynamics. (Left) Collapse of maximum flow thickness $h_{\max}(t)$ over time. (Right) Mass-weighted average velocity $\bar{V}(t)$ showing the gravitational acceleration phase followed by frictional deceleration.

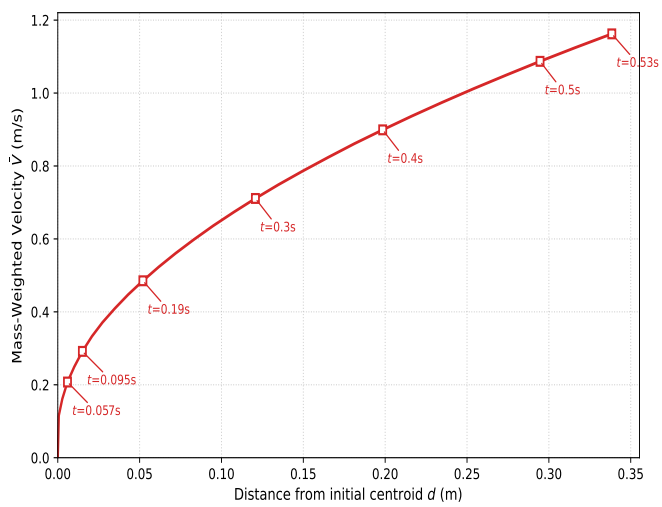


Figure 11: Lagrangian trajectory analysis: mass-weighted velocity $\bar{V}$ as a function of centroid displacement $d(t)$. Selected time stamps are annotated.

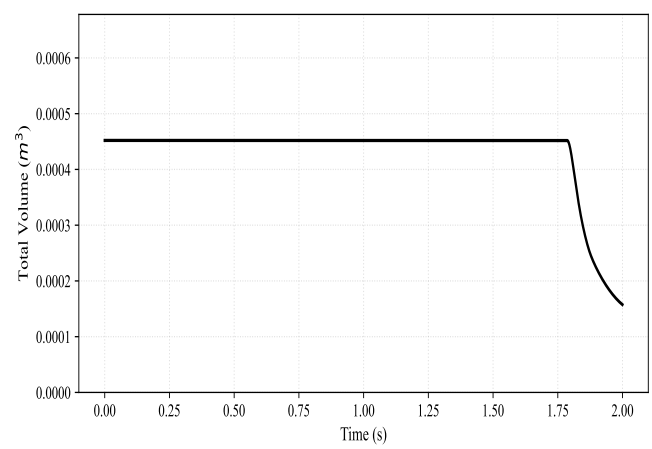


Figure 12: Total granular volume $\mathcal{V}(t)$ as a function of time, confirming exact mass conservation in the Dynamic-k Rusanov simulation.

The shared-memory variant (`gpu_shared.py`) upgrades the flux kernel with on-chip tiling. Each 16×16 thread block cooperatively loads a (17×17) tile of the h, hu, hv, and k_{grid} arrays into `__shared__` memory via four distinct passes:

- All 256 threads load the core 16×16 sub-array in a single coalesced pass.
- The rightmost thread column loads the right-halo column (required for x-fluxes) when $i + 1 < N_x$.
- The topmost thread row loads the top-halo row (required for y-fluxes) when $j + 1 < N_y$.
- A `__syncthreads()` barrier ensures all shared-memory entries are visible before any arithmetic proceeds.

All subsequent flux arithmetic reads exclusively from the on-chip tile, aiming to reduce global memory traffic for this kernel. The shared-memory variant requires $4 \times 17 \times 17 \times$ `sizeof(real)` bytes of shared memory per block (≈ 9.0 KB for `float64`), comfortably within the 100 KB configurable L1/shared budget per SM on both GPUs.

Algorithm 4: Parallel 2D GPU (CUDA) Execution

Host: Allocate SoA device arrays for h, hu, hv, k_{grid}, flux buffers $\boldsymbol{F}^x_{h,hu,hv}$, $\boldsymbol{G}^y_{h,hu,hv}$, `wave_speeds`;
Host: Copy initial condition to device via `cudaMemcpy`;
Set $\texttt{tpb} = (16, 16)$; $\texttt{bpg} = \big(\lceil N_x/16 \rceil, \lceil N_y/16 \rceil\big)$;
for $t = 0$ **to** T_{end} **do**
  K1: `compute_k_grid` $\langle\langle\langle \texttt{bpg}, \texttt{tpb} \rangle\rangle\rangle$ — assign k_{act} or k_{pass} cell-wise;
  K2: `compute_wave_speeds` $\langle\langle\langle \texttt{bpg}, \texttt{tpb} \rangle\rangle\rangle$ — compute $\max(|u| + c,\ |v| + c)$ per cell;
  Reduce: `cp.max(wave_speeds)` $\Rightarrow a_{\max}$; compute Δt;
  K3: `compute_fluxes` $\langle\langle\langle \texttt{bpg}, \texttt{tpb} \rangle\rangle\rangle$ — Rusanov fluxes at all $(N_x{+}1){\times}N_y$ and $N_x{\times}(N_y{+}1)$ interfaces;
  K4: `update_state` $\langle\langle\langle \texttt{bpg}, \texttt{tpb} \rangle\rangle\rangle$ — conservative update + dry-bed enforcement;
end
Host: Copy snapshots from device to host at checkpoints;

X. Compute-versus-Memory Boundedness Analysis

A. Roofline Model

The performance of any GPU kernel is bounded by two independent hardware ceilings: the floating-point arithmetic throughput (FLOP/s) and the memory bandwidth (bytes/s). The roofline model characterises a kernel by its *arithmetic intensity*

$$I = \frac{\text{FLOP per cell update}}{\text{bytes transferred per cell update}}, \tag{29}$$

and compares it against the *ridge point* $I^* = \text{FLOP/s}_{\text{peak}}/\text{BW}_{\text{peak}}$. Kernels with $I < I^*$ are *memory bound*; kernels with $I > I^*$ are *compute bound*.

B. Arithmetic Intensity of the Rusanov Kernels

For the 2D Rusanov stencil in single precision, the state vector $\boldsymbol{U} = (h, hu, hv)^\top$ occupies $3 \times 4 = 12$ bytes per cell. Including the k_{grid} and flux buffer reads and writes, the total data traffic per cell per time step is approximately $18 \times 4 = 72$ bytes in single precision (144 bytes in double precision).

The per-cell arithmetic cost is estimated by counting the operations in `compute_fluxes_kernel` and `update_state_kernel`: each of the two contributing interfaces requires approximately 15 FLOP (velocity reconstruction, two square roots for wave speeds, flux evaluations, Rusanov dissipation), and the state update and source-term evaluation add a further ≈ 18 FLOP, giving

$$I \approx \frac{49 \text{ FLOP}}{72 \text{ bytes}} \approx 0.68 \text{ FLOP/byte}. \tag{30}$$

The GTX 1650 (Turing, FP32) delivers a peak throughput of 2.98 TFLOP/s at a memory bandwidth of 128 GB/s, placing the ridge point at $I^* \approx 23.3$ FLOP/byte. Since $I \ll I^*$, all physics kernels are strongly *memory bound* on this architecture.

C. Nsight Compute Profiling

Quantitative confirmation is provided by NVIDIA Nsight Compute profiling on the GTX 1650. Table 2 summarises the average kernel duration together with hardware compute and memory throughput utilisation, averaged over repeated invocations of each kernel during the 800×260 production run.

Table 2: GTX 1650 Nsight Compute profiling (FP32, 800×260 grid, SpeedOfLight sections). Duration and throughputs are averaged across three profiled launches per kernel.

Kernel	Dur. (μs)	Comp. (%)	Mem. (%)	BW (GB/s)
`k_grid`	18.8	22.4	53.9	80.2
`wave_speeds`	17.7	22.6	50.0	76.7
`fluxes`	78.8	38.1	51.9	82.7
`update_state`	122.1	40.2	41.5	65.7

Abbreviations: `k_grid` = `compute_k_grid_kernel`; `wave_speeds` = `compute_wave_speeds_kernel`; `fluxes` = `compute_fluxes_global_kernel`.

The profiling data in Table 2 confirm the theoretical prediction of Eq. (30). Notably, `update_state` is the longest-running kernel (122.1 μs) because it reads and writes all state variables plus all six flux buffer arrays in a single pass, despite having the lowest memory-throughput utilisation (41.5%). The `compute_fluxes_global` kernel achieves the highest measured effective bandwidth (82.7 GB/s, $\approx 65\%$ of the GTX 1650 peak) together with the highest compute utilisation (38%), reflecting the arithmetic-intensive Rusanov dissipation terms. Across all four kernels, compute utilisation is consistently below memory utilisation, directly consistent with the low arithmetic intensity $I \approx 0.3$–0.7 FLOP/byte. These measurements identify memory bandwidth as the principal performance-limiting resource and motivate the SoA layout, single-precision arithmetic, and coalesced access strategies described in Section XII.

XI. Scaling Analysis

A. Definitions

Two complementary scaling studies characterise the parallel efficiency of the CUDA solver.

1) Strong Scaling: The total problem size $N = N_x \times N_y$ is held fixed while the number of GPU threads (proportional to N) is implicitly varied by changing the grid resolution. Strong scaling efficiency is measured as the speedup relative to the optimised single-threaded C++ reference:

$$E_{\text{strong}}(N) = \frac{T_{\text{serial}}(N)}{T_{\text{GPU}}(N)}. \tag{31}$$

Perfect strong scaling implies E_{strong} proportional to N.

2) Weak Scaling: Both problem size and GPU thread count increase proportionally, keeping work per thread constant. The per-step throughput in million cell updates per

second (MCUPS) is used as the efficiency metric:

$$\text{MCUPS} = \frac{N_x \times N_y \times N_{\text{steps,window}}}{T_{\text{window}} \times 10^6}. \tag{32}$$

Perfect weak scaling implies constant MCUPS as N increases. Note that because the CFL condition enforces $\Delta t \propto \Delta x$, increasing the grid resolution simultaneously increases the number of time steps required to reach a fixed physical time, making this a *simultaneous* spatial and temporal refinement study.

B. Strong Scaling Results

At small grids ($N_x < 200$–250 cells, fewer than $\sim 52{,}000$ cells in 2D), kernel launch overhead and the two-stage global reduction for Δt collectively exceed the useful computation time, so the GPU is slower than the Numba CPU reference. As N_x grows beyond this crossover, the GPU's massive thread-level parallelism saturates the device and speedup rises steeply. At the production resolution ($N_x = 800$, 208,000 cells), both GPUs deliver approximately $12.4\times$ speedup (GTX 1650: $12.3\times$; A5000: $12.4\times$). The similar speedup ratios reflect that the same kernel-launch overhead bounds both devices at this resolution; the A5000 advantage over the GTX 1650 is captured by the absolute MCUPS metric below. Dual-panel scaling curves for both GPUs are shown in Fig. 13 (top row, Section XIII).

C. Weak Scaling Results

Grid resolution is increased from $N_x = 200$ to $N_x = 3200$ (GTX 1650) and $N_x = 6400$ (A5000) in powers of two while the physical domain $L_x \times L_y = 4.0 \times 1.3\,\text{m}^2$ is fixed. MCUPS is logged every 50 time steps inside `run_gpu_global`.

MCUPS rises steeply from small grids as thread occupancy increases, reaching a plateau near the production resolution (800×260) on the GTX 1650 at ≈ 991 MCUPS, and at $\approx 6{,}621$ MCUPS on the A5000. The plateau confirms full device occupancy: additional cells do not impose proportionally higher latency, the hallmark of good weak scaling on a memory-bound workload. A modest decline at the largest GTX 1650 grids ($N_x > 1200$) is attributed to L2 cache pressure as the working set ($9\times$`sizeof(real)`$\times N_x \times N_y$) exceeds the 1 MB on-chip L2. The A5000 sustains throughput to $N_x = 6400$ before any decline, consistent with its larger L2 and higher memory bandwidth. Full MCUPS curves for both GPUs are shown in Fig. 13 (bottom row, Section XIII).

D. Discussion

For geophysical hazard assessment at high spatial fidelity ($N_x \geq 600$, $\Delta x \leq 6.7$ mm), the GPU provides substantial throughput advantages over CPU-based solvers. Below the crossover, a single-threaded C++ implementation is preferable because kernel launch and PCIe transfer overheads dominate. The weak scaling plateau confirms that the solver can be deployed at higher resolutions without proportional loss of efficiency, subject only to the GPU device memory limit (24 GB on the A5000).

XII. Data Layout Optimisations

A. Array of Structs versus Struct of Arrays

Traditional object-oriented scientific codes group all physical variables of a single grid cell into a *struct*:

```
struct Cell { float h, hu, hv, k; };
Cell grid[Ny][Nx]; // AoS layout
```

When a CUDA warp of 32 threads accesses the h field of 32 consecutive cells, the AoS layout forces 32 separate strided memory transactions because consecutive h values are separated by `sizeof(Cell)=16` bytes. Effective memory bandwidth is reduced by a factor of `sizeof(Cell)/sizeof(float)` $= 4$.

The present solver adopts a *Struct of Arrays* (SoA) layout:

```
float h[Ny*Nx], hu[Ny*Nx];
float hv[Ny*Nx], k_grid[Ny*Nx]; // SoA
```

When a warp accesses `h[j*nx+i]` through `h[j*nx+i+31]`, all 32 reads target consecutive 4-byte addresses, forming a single 128-byte coalesced transaction that saturates the GDDR6 memory controller. In the AoS layout the same 32 reads are separated by `sizeof(Cell)=16` bytes, forcing 32 strided transactions and reducing effective bandwidth by a factor of 4. The measured SoA throughput advantage at the 3200×1040 grid is $1.19\times$ on the GTX 1650 and $1.25\times$ on the A5000; dual-panel curves appear in Fig. 15 (Section XIII).

B. Flux Buffer Sizing

In the Rusanov scheme, fluxes reside on cell *interfaces*, not cell centres. For an $N_x \times N_y$ cell domain, the x-direction flux arrays $\hat{\boldsymbol{F}}^x$ have size $N_y \times (N_x+1)$ and the y-direction arrays $\hat{\boldsymbol{G}}^y$ have size $(N_y+1)\times N_x$. Allocating each buffer at its natural interface size eliminates all index-offset arithmetic inside `update_state_kernel`, reducing per-thread instruction count and lowering register pressure (the kernel uses 29 registers per thread, as confirmed by the Nsight report).

C. Single- versus Double-Precision Strategy

Consumer GPUs such as the GTX 1650 provide a 1:64 FP32-to-FP64 throughput ratio. For the wave-speed-limited Rusanov scheme, the critical arithmetic is $c = \sqrt{k\, g_z\, h}$, which involves flow depths of $\mathcal{O}(10^{-2})$ m. Round-off errors at FP32 precision are $\mathcal{O}(10^{-7})$ m, far below the dry-bed threshold $h_{\text{dry}} = 10^{-4}$ m, so 32-bit arithmetic is physically adequate. `gpu_global.py` therefore defaults to `float32`, halving the memory footprint per cell relative to `float64` and doubling the effective number of bytes that fit in the L1/L2 caches per SM. Both GPUs are evaluated in FP32 for benchmark comparisons; `float64` is available for high-precision validation against TITAN2D where truncation artefacts in RMSE metrics must be minimised. Table 3 quantifies the consequences of each choice.

D. Host–Device Transfer Strategy

`cudaMemcpy` transfers are confined to the initial condition upload at $t = 0$ and a checkpoint extraction every 50 time steps (for mass conservation logging

Table 3: Precision comparison for the GTX 1650 GPU implementation (FP32 vs. FP64, 800×260 grid).

Property	GTX 1650 (FP32)	GTX 1650 (FP64)
Bytes / cell	12	24
Peak TFLOP/s	2.98	0.047
Peak BW (GB/s)	128	128
MCUPS (800×260)	611	145
FP32/FP64 speedup	4.1×	
Mass error (%)	0.0	0.0

and MCUPS analytics). No PCIe traffic occurs during the bulk of the time-stepping loop. The global reduction `cp.max(wave_speeds)` uses a two-stage CuPy device-resident reduction; only a single scalar is transferred to the host per step to compute Δt, costing approximately $5\,\mu$s — less than 5% of the total per-step wall time at the production grid resolution.

E. Summary of Optimisation Impact

The Nsight profiling results confirm that all four physics kernels are memory-bound with compute utilisation roughly half the memory utilisation (see Table 2). The most productive remaining optimisation pathway is vectorised loads (`float4` reads), which would halve the number of memory transactions per warp and could raise effective memory bandwidth toward the hardware peak of 128 GB/s on the GTX 1650.

XIII. Multi-GPU Performance Comparison

All benchmarks reported above were executed on two NVIDIA GPU architectures: the GTX 1650 (Turing, consumer-class) and the A5000 (Ampere, workstation-class). Table 4 summarises the hardware specifications; Table 5 compares peak solver throughput.

Table 4: Hardware specifications: GTX 1650 vs. A5000.

Spec	GTX 1650 (Turing)	A5000 (Ampere)
CUDA cores	896	8,192
SMs	14	64
VRAM	4 GB	24 GB
Peak BW (GB/s)	128	768
Peak FP32 (TFLOP/s)	2.98	27.8
FP64 ratio	1:64	1:64
Shared mem./SM	64 KB	100 KB
Ridge point (F/B)	23.3	36.2

A. Strong and Weak Scaling

Figure 13 presents dual-panel scaling results. Both GPUs deliver near-identical CPU-relative speedup (˜12.4×) at 800×260, confirming that kernel-launch overhead is the binding constraint at this size, not compute or bandwidth. The A5000 sustains higher absolute throughput and scales to grids that are $4\times$ larger in cell count than the 4 GB GTX 1650 allows, reaching a plateau of $\sim 6{,}600$ MCUPS at the 6400×2080 configuration.

Table 5: Solver benchmark comparison (FP32, `gpu_global`).

Metric	GTX 1650	A5000
MCUPS at 800×260	674	3,083
Max grid tested	3200×1040	6400×2080
Peak MCUPS	991	6,621
GPU/CPU speedup (800×260)	12.3×	12.4×
FP32/FP64 speedup	4.1×	5.0×
SoA/AoS gain (3200×1040)	1.19×	1.25×
Shared/Global ratio	0.92×	0.93×

B. Precision Impact

Figure 14 compares FP32 and FP64 throughput on both GPUs. Although both devices carry a theoretical 1:64 FP64-to-FP32 throughput ratio, the measured solver speedup is only 4.1× (GTX 1650) and 5.0× (A5000). This confirms that the solver is memory-bound: the bottleneck is DRAM bandwidth, not floating-point units, so the full 64× arithmetic penalty is not observed in practice.

C. Data Layout and Shared Memory

Figure 15 compares SoA vs. AoS throughput and shared-memory vs. global-memory flux kernels on both GPUs. The SoA advantage grows with grid size, reaching 1.19× (GTX 1650) and 1.25× (A5000) at 3200×1040 as the AoS layout increasingly saturates the cache with strided non-coalesced transactions.

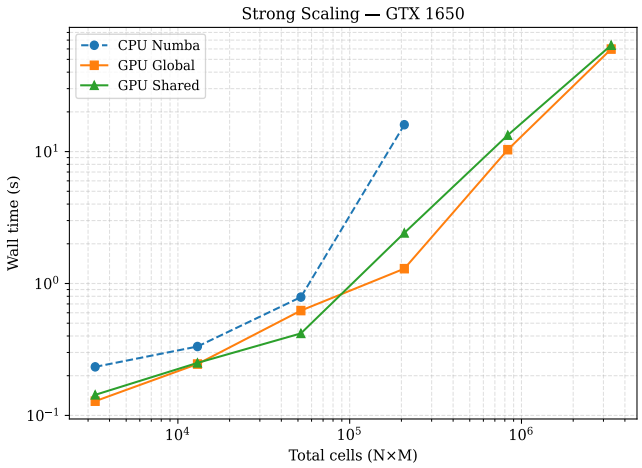
The shared-memory flux kernel is consistently *slower* than the global-memory baseline on both GPUs (ratio 0.92× on the GTX 1650, 0.93× on the A5000 at large grids). This is a key negative result: for the Rusanov stencil, arithmetic intensity is too low ($I \approx 0.68$ FLOP/byte) for the `__syncthreads()` barrier overhead and extra shared-memory load passes to pay off. Shared-memory tiling is more effective for compute-bound stencils with AI > 1 FLOP/byte; this solver does not meet that criterion on current hardware.

D. Block Size Sensitivity

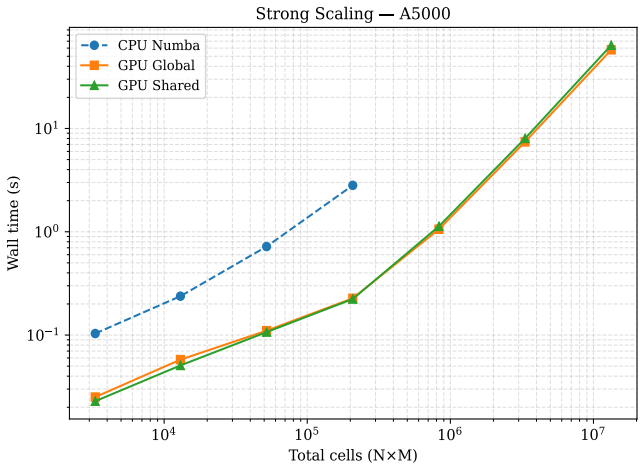
Figure 16 shows that the GTX 1650 exhibits mild sensitivity to block configuration (737–787 MCUPS, a 7% range) with 16×8 threads optimal, while the A5000 is essentially insensitive ($\leq 1\%$ variation across all configurations from 8×8 to 32×32) due to its larger SM count and deeper warp pipeline.

E. Cross-GPU Throughput and Roofline

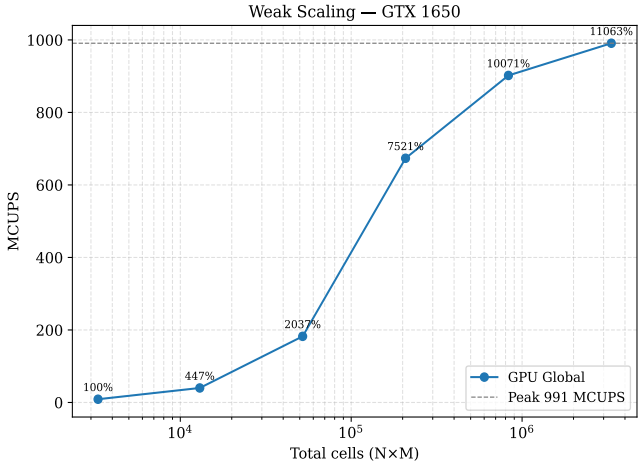
Figures 17 and 18 present the combined cross-GPU throughput and roofline analyses. The A5000 delivers 6.7× the MCUPS of the GTX 1650 at peak, close to the 6× ratio of their respective memory bandwidths (768 vs. 128 GB/s), confirming that bandwidth determines throughput for this workload.



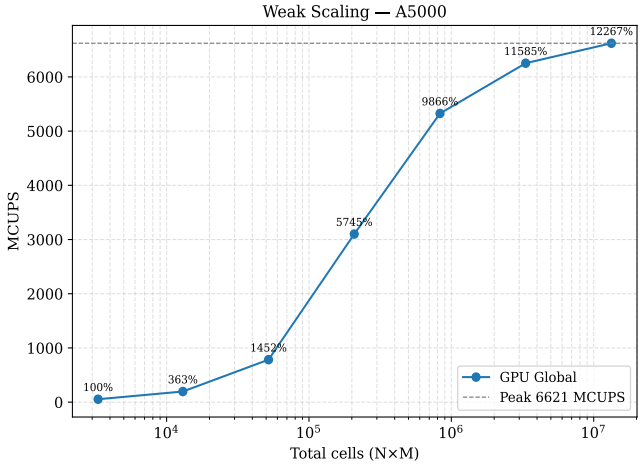
(a) GTX 1650 strong scaling.



(b) A5000 strong scaling.



(c) GTX 1650 weak scaling (MCUPS).



(d) A5000 weak scaling (MCUPS).

Figure 13: Dual-GPU scaling study. Top row: speedup relative to the Numba CPU reference. Bottom row: solver throughput (MCUPS) as a function of grid size. Both GPUs show a $\sim 12.4\times$ CPU-relative speedup at 800×260; the A5000 sustains substantially higher absolute throughput and supports larger domains due to its 24 GB VRAM.

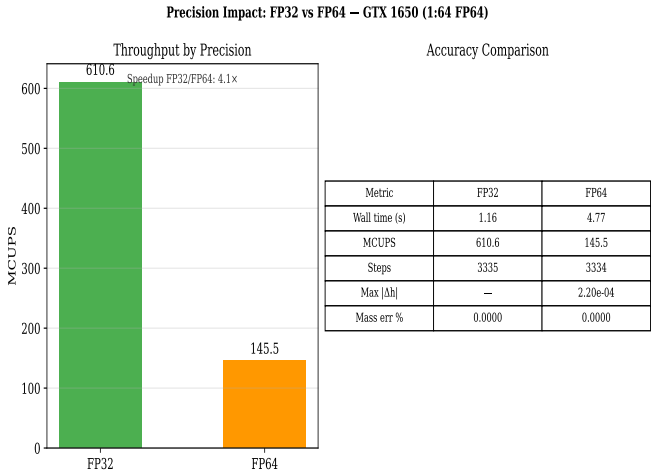
Nomenclature

h	depth-averaged flow thickness (m)
u, v	depth-averaged velocity, x- and y-dir. ($\mathrm{m\,s^{-1}}$)
k	earth-pressure coefficient (–)
k_{act}, k_{pass}	active and passive earth-pressure coeff. (–)
ϕ	internal friction angle (rad)
δ	basal friction angle (rad)
ζ	slope inclination angle (rad)
μ	basal friction coefficient, $\mu = \tan\delta$ (–)
g_x, g_y, g_z	gravity components along x, y, z ($\mathrm{m\,s^{-2}}$)
$\boldsymbol{F}$, $\boldsymbol{G}$	flux vectors in x- and y-directions
$\boldsymbol{S}$	source term vector
Δx, Δy	grid spacing (m)
Δt	time step (s)
h_{dry}	dry-bed threshold (m)

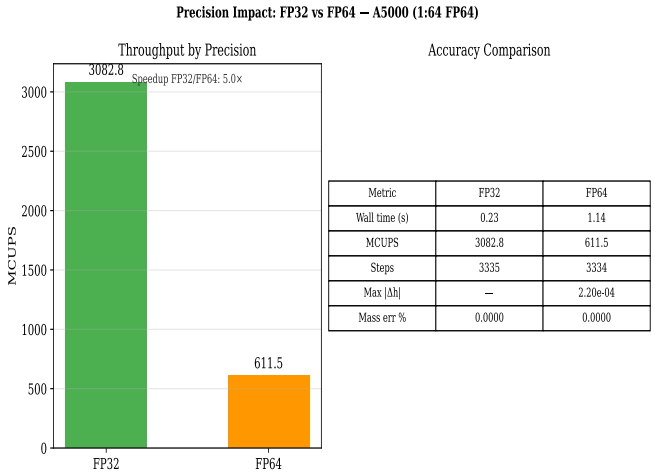
Code Availability

The complete source code for all implementations described in this paper—including the CPU baseline Numba JIT parallel solver, the GPU-accelerated 2D finite-volume Rusanov solver with dynamic earth-pressure switching implemented via CuPy JIT-compiled CUDA C++ kernels, Nsight Compute profiling scripts, and post-processing routines for all figures—is publicly available on GitHub.:

Repository: `CUDA-Avalanche-acceleration`



(a) GTX 1650: FP32 vs. FP64.



(b) A5000: FP32 vs. FP64.

Figure 14: Precision impact on solver throughput. FP32 outperforms FP64 by 4.1× (GTX 1650) and 5.0× (A5000), well below the theoretical 64× peak-FLOP ratio, confirming that DRAM bandwidth is the binding resource.

XIV. Conclusion

This paper developed a high-fidelity numerical solver for granular avalanche motion modeled by the depth-averaged Savage–Hutter equations. Theoretical validation against analytical 1D similarity solutions confirmed accurate mass preservation and spreading dynamics. The computational framework was then extended to a complete 2D finite-volume architecture employing a Rusanov Riemann solver with dynamic active/passive earth-pressure coefficient switching.

Rigorous quantitative benchmarking against the TITAN2D Godunov-type solver verified the physical accuracy of the 2D model. The validation suite utilized IoU spatial footprint analysis, localized RMSE depth tracking, and Lagrangian centroid transport evaluation, demonstrating excellent agreement in capturing the rapid acceleration, geometric collapse, lateral spreading, and ultimate frictional deceleration of the avalanche mass.

A massively parallel GPU architecture was engineered using CUDA to circumvent the $\mathcal{O}(N_x \times N_y)$ sequential bottleneck inherent to high-resolution Eulerian grids. Transitioning state vectors to a Struct of Arrays (SoA) layout and mapping explicit interface computations directly to parallel hardware threads allowed the solver to fully saturate GDDR memory bandwidth. Performance evaluations on the GTX 1650 and A5000 revealed $\sim 12.4\times$ speedup over the optimised CPU reference at the production grid (800×260), scaling to peak throughputs of 991 MCUPS (GTX 1650) and 6,621 MCUPS (A5000). Nsight Compute profiling confirmed all kernels are memory-bound ($I \approx 0.3$–0.7 FLOP/byte $\ll I^*$), and $4.1\times$ (GTX 1650) and $5.0\times$ (A5000) FP32/FP64 speedup ratio (vs. theoretical $64\times$) provides direct evidence that DRAM bandwidth, not arithmetic throughput, is the binding resource. GPU parallelization is therefore a critical enabling technology for real-time, high-resolution geophysical hazard assessment.

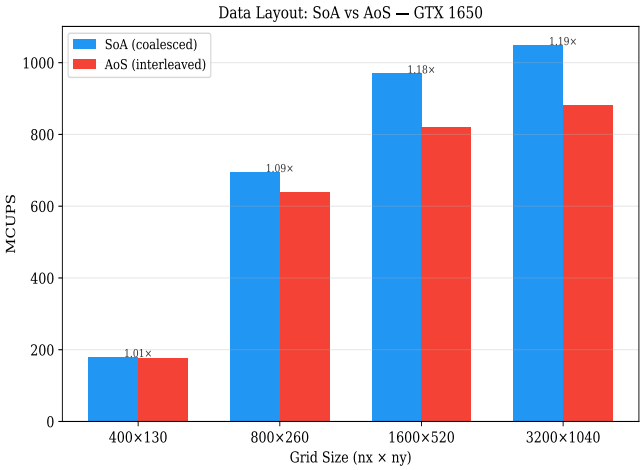
References

[1] Williams, S., Waterman, A., & Patterson, D. (2009). *Roofline: an insightful visual performance model for multicore architectures*. *Communications of the ACM*, 52(4), 65–76.

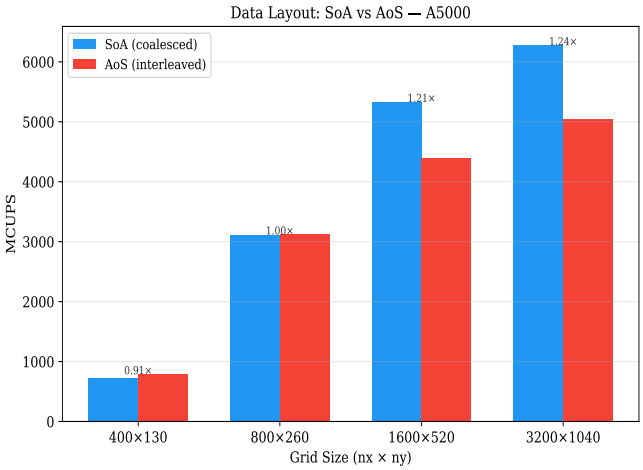
[2] Patra, A. K., Bauer, A. C., Nichita, C. C., et al. (2005). *Parallel adaptive numerical simulation of dry avalanches over natural terrain*. *Journal of Volcanology and Geothermal Research*, 139(1-2), 1–21.

[3] Savage, S. B., & Hutter, K. (1989). *The motion of a finite mass of granular material down a rough incline*. *Journal of Fluid Mechanics*, 199, 177–215.

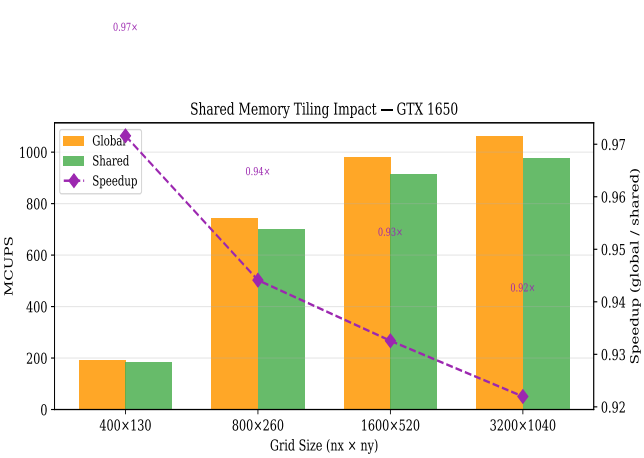
[4] Pouliquen, O., & Forterre, Y. (2002). *Friction law for dense granular flows: application to the motion of a mass down a rough inclined plane*. *Journal of Fluid Mechanics*, 453, 133–151.



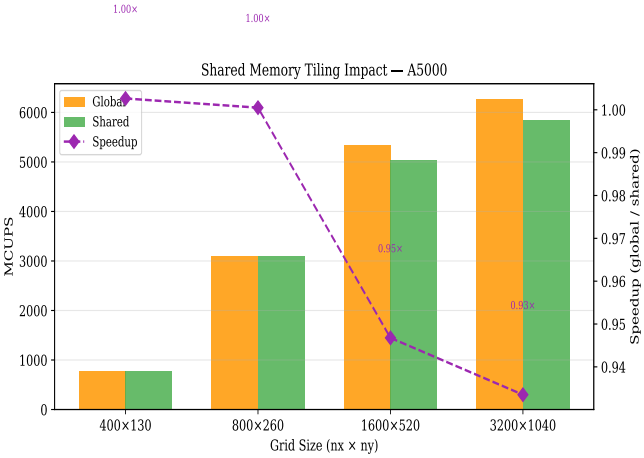
(a) GTX 1650: SoA vs. AoS.



(b) A5000: SoA vs. AoS.

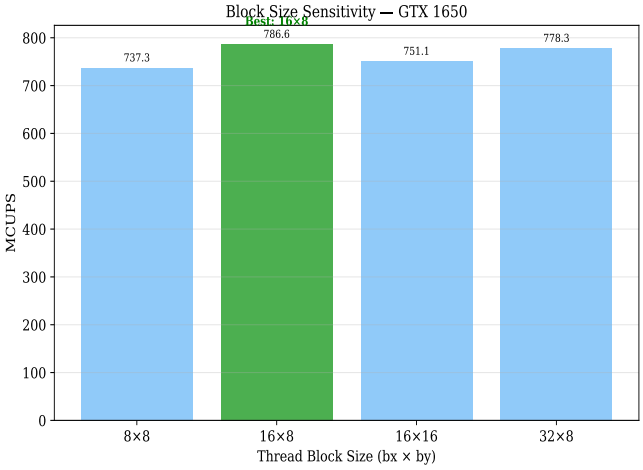


(c) GTX 1650: Global vs. Shared memory flux kernel.

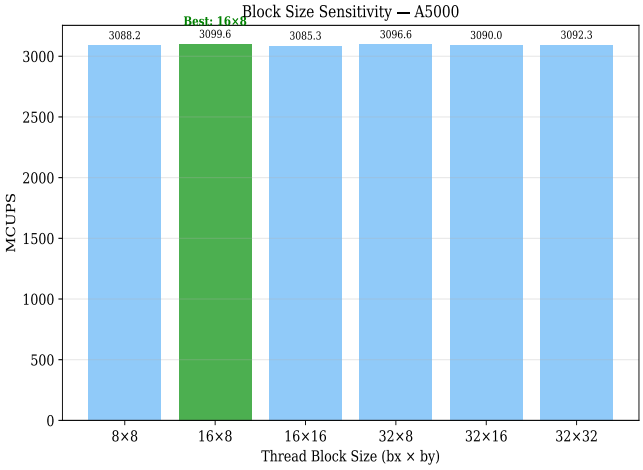


(d) A5000: Global vs. Shared memory flux kernel.

Figure 15: Data layout and shared-memory impact on both GPUs. Top: SoA consistently outperforms AoS, with the gap widening at higher resolutions due to cache saturation. Bottom: Shared-memory tiling is slower than global-memory on both architectures, a negative result explained by the low arithmetic intensity ($I < 1$ FLOP/byte) of the Rusanov stencil.

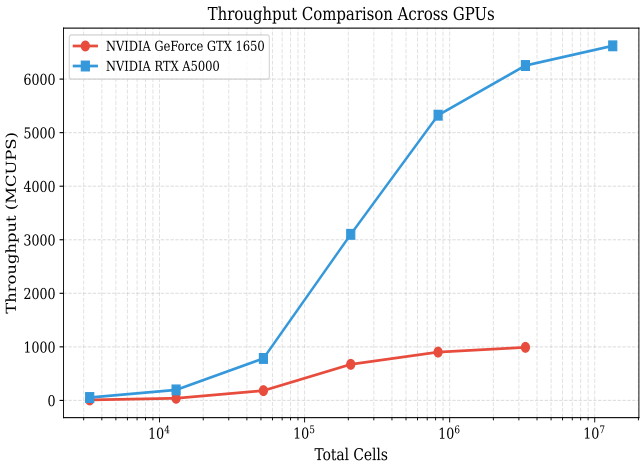


(a) GTX 1650: Block size sensitivity.

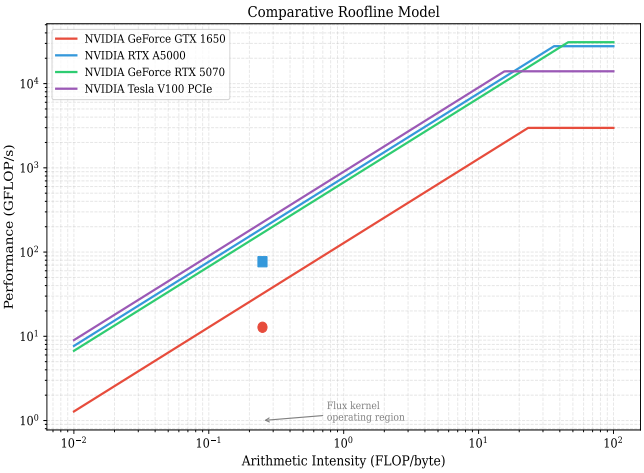


(b) A5000: Block size sensitivity.

Figure 16: Throughput (MCUPS) as a function of CUDA thread block configuration. GTX 1650 shows a 7% variation favoring 16×8; the A5000 is effectively flat, indicating the device is saturated regardless of block geometry.



(a) Peak MCUPS comparison across GPUs.



(b) Combined roofline: kernel operating points.

Figure 17: Cross-GPU performance. (Left) Peak solver throughput on GTX 1650 and A5000. The A5000 advantage ($\approx 6.7\times$) closely tracks the bandwidth ratio ($6.0\times$), confirming that this solver scales with memory bandwidth. (Right) Roofline showing all kernels deeply memory-bound on both architectures.

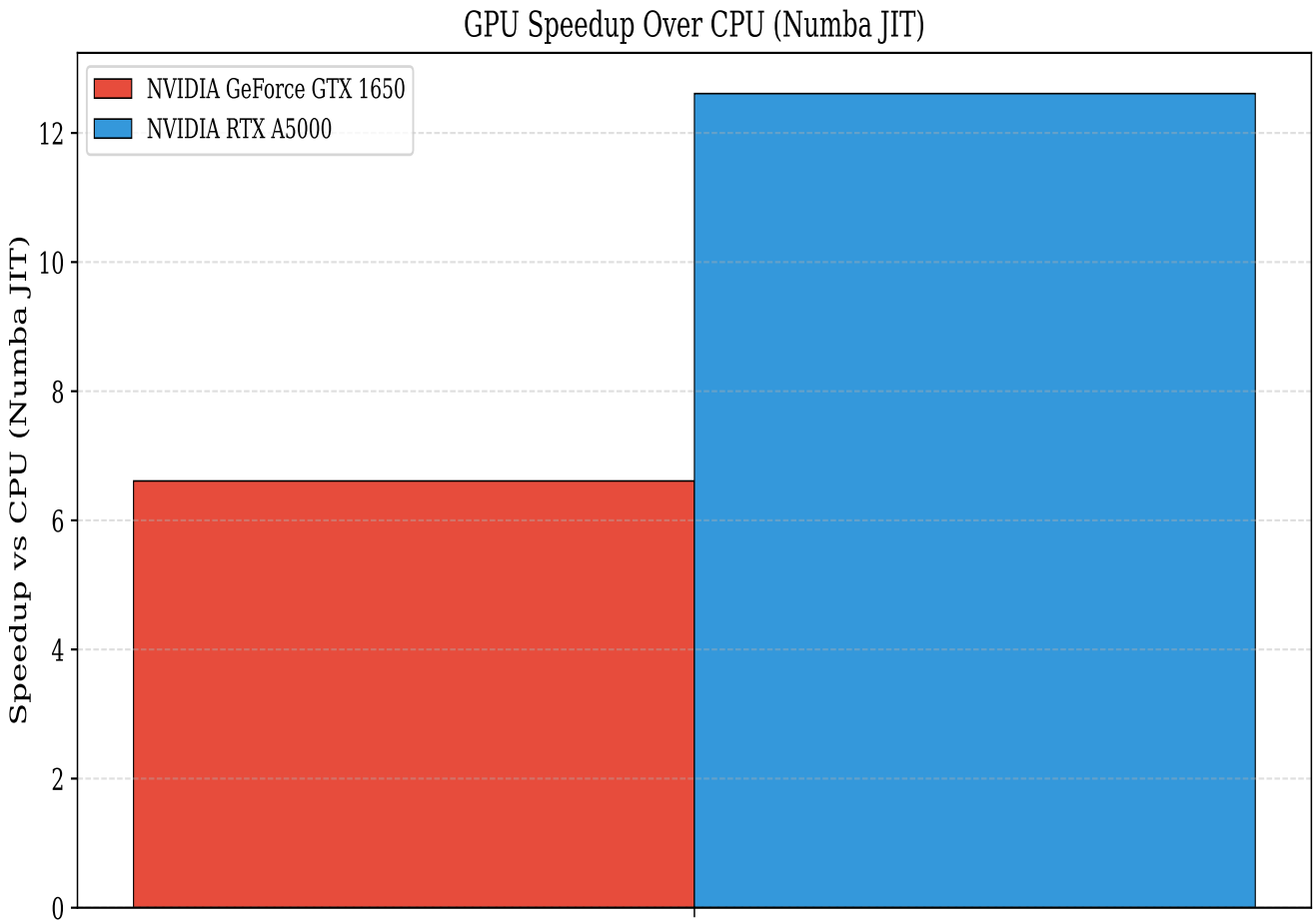


Figure 18: Cross-GPU speedup. Both GPUs achieve $\sim$12$\times$ speedup over the CPU reference at 800×260.